Имитационное моделирование

Лабораторная работа №4. Реализация основных моделей в агентном подходе

Ведьмина Александра Сергеевна

Содержание

1 Цель работы **7**

2 Задание **8**

3 Теоретическое введение **9**
3.1 Агентное моделирование 9
3.2 Классическая SIR-модель и её агентная версия 9
3.3 Репродуктивное число и эпидемический порог 10
3.4 Литературное программирование и воспроизводимость 11
3.5 Основные параметры модели 11

4 Выполнение лабораторной работы **13**
4.1 Подготовка окружения 13
4.2 Структура проекта 15
4.3 Реализация модели 16
4.4 Базовый эксперимент 18
4.5 Исследование коэффициента заразности 19
4.6 Исследование миграции 21
4.7 Сценарные эксперименты 23
4.8 Оптимизация и итоговая комплексная визуализация 28
4.9 Литературное программирование и производные форматы 30

5 Дополнительные задания **34**
5.1 Задание 1. Базовый анализ `R_0` 34
5.2 Задание 2. Порог эпидемии 34
5.3 Задание 3. Эффект гетерогенности 35
5.4 Задание 4. Влияние миграции 39
5.5 Задание 5. Карантинные меры 39
5.6 Задание 6. Оптимизация при ограничении на пик 42
5.7 Выполнение Jupyter notebook 43

6 Полный код проекта **48**
6.1 Исходный код модели `sir_model.jl` 48
6.2 Исходный код аналитики `sir_analysis.jl` 52
6.3 Генератор literate-форматов `generate.jl` 65
6.4 Скрипт базового эксперимента `sir_run_basic.jl` 67

6.5 Скрипт сканирования beta sir_scan_beta.jl 67
6.6 Скрипт исследования миграции sir_migration_effect.jl 68
6.7 Скрипт итоговой визуализации sir_visualize_dynamics.jl 68
6.8 Скрипт порогового исследования sir_threshold_study.jl 69
6.9 Скрипт исследования гетерогенности sir_heterogeneity_effect.jl 69
6.10 Скрипт карантинного сценария sir_quarantine.jl 70
6.11 Скрипт оптимизации sir_optimize_parameters.jl 70
6.12 Literate-источник sir_literate.jl 71
6.13 Literate-источник sir_param_literate.jl 72
6.14 Literate-источник sir_threshold_literate.jl 73
6.15 Literate-источник sir_heterogeneity_literate.jl 74
6.16 Literate-источник sir_quarantine_literate.jl 75
6.17 Literate-источник sir_optimize_literate.jl 76
6.18 Тесты runtests.jl 77

7 Выводы **79**

Список литературы **80**

Список иллюстраций

4.1 Запуск Julia и создание проекта lab_04_models 14
4.2 Активация проекта через Pkg.activate 15
4.3 Установка пакетов проекта 15
4.4 Запуск базового сценария 18
4.5 Базовая динамика агентной SIR-модели 19
4.6 Запуск исследования коэффициента beta 20
4.7 Фрагмент файла beta_scan_all.csv 20
4.8 Сканирование коэффициента заразности 21
4.9 Запуск скрипта sir_migration_effect.jl 22
4.10 Влияние интенсивности миграции на время и масштаб пика 22
4.11 Сценарий baseline 24
4.12 Сценарий heterogeneous_beta 25
4.13 Сценарий fast_migration 26
4.14 Сценарий slow_detection 27
4.15 Сравнение сценариев по пику и смертности 28
4.16 Запуск скрипта sir_visualize_dynamics.jl 29
4.17 Комплексная визуализация зависимости метрик от beta 29
4.18 Генерация производных форматов через generate.jl 31
4.19 Повторная генерация literate-артефактов после расширения проекта . 32

5.1 Запуск порогового исследования 34
5.2 Пороговое исследование по коэффициенту beta 35
5.3 Запуск сценария гетерогенности 36
5.4 Сравнение глобальной динамики для однородного и неоднородного сценариев 36
5.5 Покомпонентная динамика заражённых по городам 37
5.6 Сводная визуализация эффекта гетерогенности 38
5.7 Запуск дополнительного анализа миграции 39
5.8 Запуск карантинного сценария 39
5.9 Сравнение карантинных режимов по пику и смертности 40
5.10 Сравнение лучшего карантинного режима с базовым по временной динамике 41
5.11 Запуск сценария оптимизации 42
5.12 Фронт допустимых решений при ограничении на пик 43
5.13 Открытая папка notebooks/ в Jupyter 44
5.14 Notebook sir_heterogeneity.ipynb 45

5.15 Notebook `sir_optimize.ipynb` 45
5.16 Notebook `sir_param.ipynb` 46
5.17 Notebook `sir_quarantine.ipynb` 46
5.18 Notebook `sir_threshold.ipynb` 47
5.19 Notebook `sir.ipynb` 47

Список таблиц

3.1 Параметры базовой агентной SIR-модели 11

1 Цель работы

Изучить реализацию эпидемиологической модели SIR в агентном подходе на языке Julia с использованием фреймворка Agents.jl, организовать воспроизводимый проект в структуре DrWatson, выполнить базовые и дополнительные вычислительные эксперименты, подготовить literate-версии скриптов и собрать отчёт с полным описанием полученных результатов.

2 Задание

В рамках лабораторной работы требовалось:

1. создать отдельный проект `DrWatson` для агентной SIR-модели;
2. подключить необходимые Julia-пакеты и подготовить структуру каталогов;
3. реализовать агентную модель распространения инфекции на графе городов;
4. выполнить базовый эксперимент, сканирование коэффициента заразности, исследование миграции и сценарные расчёты;
5. оформить основные сценарии в стиле literate programming и сгенерировать из них производные форматы `.jl`, `.ipynb` и `.qmd`;
6. выполнить код из Jupyter notebook и включить результаты в отчёт;
7. выполнить дополнительные задания: оценку `R_0`, поиск эпидемического порога, исследование гетерогенности, анализ миграции, карантинные меры и оптимизацию при ограничении на пик заболеваемости.

3 Теоретическое введение

3.1 Агентное моделирование

Агентное моделирование (Agent-Based Modeling, ABM) описывает систему как совокупность автономных сущностей, каждая из которых имеет собственное состояние и действует по локальным правилам [1; 8]. Глобальное поведение в ABM не задаётся напрямую, а возникает из большого числа локальных взаимодействий. Для эпидемиологических задач это особенно удобно, потому что позволяет моделировать:

- отдельных людей, а не только доли популяции;
- неоднородность по городам и сценариям;
- миграцию между узлами графа;
- логические меры управления, например карантин.

В работе для реализации ABM используется пакет Agents.jl [3], ориентированный на построение агентных моделей на графах, решётках и других пространствах.

3.2 Классическая SIR-модель и её агентная версия

Классическая модель SIR делит популяцию на три класса [5; 6]:

- `S` — восприимчивые;
- `I` — инфицированные;

- R — выздоровевшие.

В непрерывной постановке динамика описывается системой дифференциальных уравнений. В данной лабораторной работе используется дискретная агентная постановка, где каждый человек моделируется как отдельный агент, а переходы между состояниями реализуются как последовательность событий:

1. агент перемещается между городами;
2. инфицированный агент заражает случайное число контактов;
3. по достижении `infection_period` агент либо выздоравливает, либо умирает;
4. выздоровевший агент может быть повторно инфицирован с заданной вероятностью.

Модель строится на полном графе из трёх вершин, где каждая вершина соответствует городу. Это делает постановку одновременно простой и наглядной: можно отдельно анализировать влияние `beta`, миграции, неоднородности по городам и карантинного ограничения.

3.3 Репродуктивное число и эпидемический порог

При фиксированной длительности болезни `infection_period` теоретическая оценка базового репродуктивного числа имеет вид

$$R_0 \approx \beta \cdot infection_period.$$

В базовом сценарии `beta_und = 0.5`, `infection_period = 14`, поэтому

$$R_0 \approx 0.5 \cdot 14 = 7.$$

Такое значение существенно больше единицы, а значит, режим должен соответствовать устойчивому развитию эпидемии.

3.4 Литературное программирование и воспроизводимость

В работе использовалась методология literate programming [7], при которой код и объясняющий текст ведутся из единого источника. Пакет Literate.jl [4] позволяет из одного `.jl`-файла получить:

- чистый Julia-скрипт;
- Jupyter notebook;
- Quarto-документацию.

Для организации воспроизводимого исследовательского проекта использован пакет DrWatson [2]. Он задаёт стандартную структуру каталогов и упрощает связь между исходным кодом, данными, графиками и документами.

3.5 Основные параметры модели

Основные параметры агентной SIR-модели приведены в табл. 3.1.

Таблица 3.1: Параметры базовой агентной SIR-модели

Параметр	Обозначение	Значение	Интерпретация
Размерности городов	`Ns`	`[1000, 1000, 1000]`	население трёх городов
Заразность до выявления	`beta_und`	`[0.5, 0.5, 0.5]`	интенсивность передачи инфекции
Заразность после выявления	`beta_det`	`[0.05, 0.05, 0.05]`	ослабленная передача после обнаружения

Параметр	Обозначение	Значение	Интерпретация
Длительность инфекции	`infection_period`	14	число шагов болезни
Время выявления	`detection_time`	7	момент переключения на `beta_det`
Смертность	`death_rate`	0.02	вероятность смерти при завершении болезни
Повторное заражение	`reinfection_probability`	0.1	вероятность повторного перехода `R -> I`
Начальные заражённые	`Is`	`[0, 0, 1]`	источник эпидемии в третьем городе
Базовое число шагов	`n_steps`	100	горизонт моделирования

4 Выполнение лабораторной работы

4.1 Подготовка окружения

4.1.1 Запуск Julia и создание проекта

Подготовка проекта началась с запуска Julia REPL, подключения DrWatson и создания каталога `lab_04_models` с типовой структурой воспроизводимого проекта.

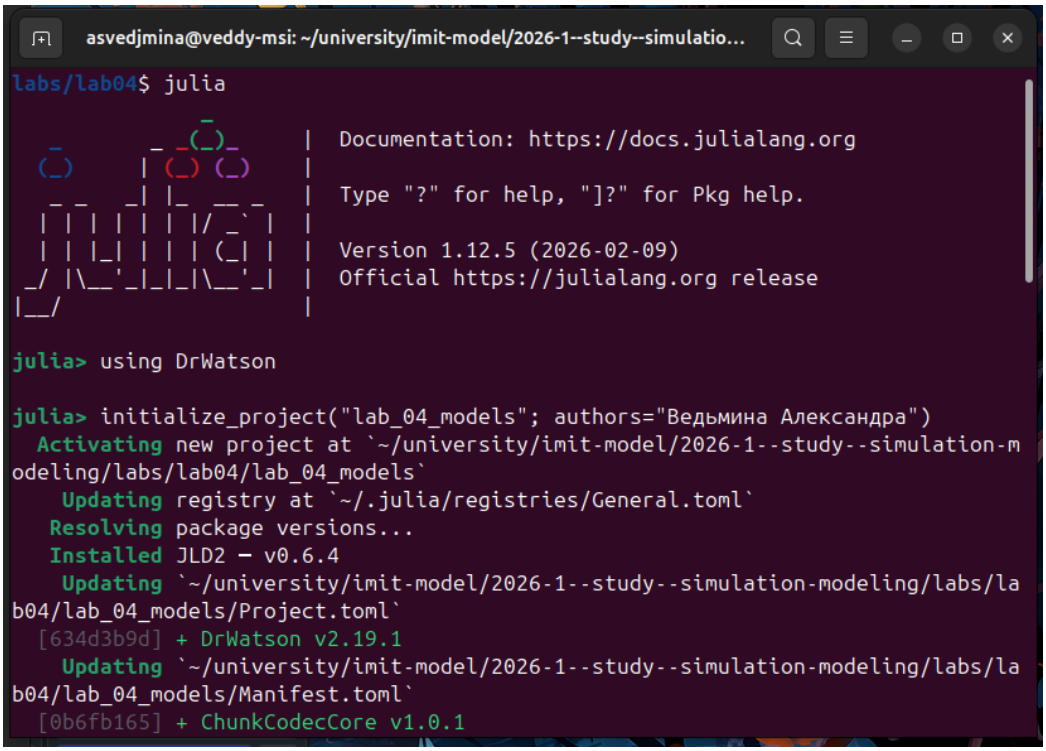

Рисунок 4.1: Запуск Julia и создание проекта `lab_04_models`

На этом этапе DrWatson создал каркас каталогов `src/`, `scripts/`, `docs/`, `notebooks/`, `data/`, `plots/`, `test/` и файл `Project.toml`, куда затем были добавлены зависимости проекта.

4.1.2 Активация окружения и установка пакетов

После создания проекта окружение было активировано через `Pkg.activate`, а затем в него были установлены необходимые пакеты: `Agents`, `DataFrames`, `Plots`, `CSV`, `JLD2`, `Literate`, `IJulia`, `Quarto`, `StatsBase`, `Distributions` и другие.

```
julia> using Pkg

julia> Pkg.activate("lab_04_models/")
  Activating project at `~/university/imit-model/2026-1--study--simulation-model
ing/labs/lab04/lab_04_models`

julia>

julia>
```

Рисунок 4.2: Активация проекта через `Pkg.activate`

```
asvedjmina@veddy-msi: ~/university/imit-model/2026-1--study--simulatio...

julia> Pkg.add([ "Agents", "DataFrames", "Plots", "CSV", "JLD2", "Literate", "IJ
ulia", "Quarto", "BlackBoxOptim", "StatsBase", "Distributions"])
   Resolving package versions...
   Installed libpng_jll ──────── v1.6.56+0
   Installed FreeType2_jll ────── v2.14.3+1
   Installed Accessors ────────── v0.1.44
   Installed SpecialFunctions ─── v2.7.2
   Installed Cairo_jll ────────── v1.18.6+0
   Installed StructArrays ─────── v0.7.3
   Installed PrettyTables ─────── v3.3.2
   Installed SpatialIndexing ──── v0.1.6
   Installed BlackBoxOptim ────── v0.6.4
   Installed Agents ───────────── v7.0.1
  Installing artifacts ═══════════════════════════════════════ 3/3
    Updating `~/university/imit-model/2026-1--study--simulation-modeling/labs/la
b04/lab_04_models/Project.toml`
  [46ada45e] + Agents v7.0.1
  [a134a8b2] + BlackBoxOptim v0.6.4
  [336ed68f] + CSV v0.10.16
  [a93c6f00] + DataFrames v1.8.1
  [31c24e10] + Distributions v0.25.123
  [7073ff75] + IJulia v1.34.4
  [033835bb] + JLD2 v0.6.4 [loaded: v0.6.3]
```

Рисунок 4.3: Установка пакетов проекта

Фактический состав зависимостей проекта определяется файлом `Project.toml`. Для воспроизводимости дальнейших запусков это важно, поскольку все скрипты выполняются с явным указанием `--project=labs/lab04/lab_04_models`.

4.2 Структура проекта

После подготовки окружения была сформирована следующая структура:

- `src/sir_model.jl` — базовая агентная модель и шаг агента;
- `src/sir_analysis.jl` — функции для проведения экспериментов, построения таблиц и графиков;
- `scripts/` — исполняемые сценарии и literate-источники;
- `docs/` — автоматически сгенерированные Quarto-документы;
- `notebooks/` — сгенерированные Jupyter notebook;
- `data/` — численные результаты экспериментов;
- `plots/` — итоговые графики;
- `test/runtests.jl` — проверка корректности логики модели.

Такое разбиение полностью повторяет подход из лабораторной работы №3 и хорошо согласуется с рекомендациями DrWatson.

4.3 Реализация модели

4.3.1 Агент и пространство модели

Основная модель реализована в файле `src/sir_model.jl`. Агент задаётся структурой `Person`, содержащей число дней болезни, текущий статус и домашний город:

```
@agent struct Person(GraphAgent)
    days_infected::Int
    status::Symbol
    home_city::Int
end
```

Пространство модели — полный граф из трёх вершин. Каждая вершина соответствует отдельному городу, а потому агент всегда находится в одном из трёх дискретных узлов графа.

4.3.2 Инициализация модели

Функция `initialize_sir` создаёт полный граф, размещает в каждом городе заданное число агентов, задаёт матрицу миграции и заражает стартовое число агентов. Если матрица миграции не передана явно, она создаётся автоматически.

В базовой постановке используется симметричный сценарий по населению и по коэффициентам передачи, однако в дополнительных заданиях параметры изменяются по сценариям.

4.3.3 Логика шага агента

Один шаг агента реализуется функцией `sir_agent_step!` и состоит из четырёх частей:

1. `migrate!` — возможный переход в другой город;
2. `transmit!` — заражение части контактов в текущем городе;
3. увеличение счётчика `days_infected`;
4. `recover_or_die!` — выздоровление или смерть.

Для заражения используется распределение Пуассона, поэтому число новых контактов в каждом шаге является случайной величиной. До момента `detection_time` используется `beta_und`, а затем — более слабая интенсивность `beta_det`.

4.3.4 Карантин и вспомогательные функции

В модель дополнительно встроены:

- `create_migration_matrix` для явного задания интенсивности миграции;
- `infected_count`, `recovered_count`, `susceptible_count` и другие функции подсчёта;
- `lock_city!` и `update_quarantine!` для сценариев с ограничением миграции.

Таким образом, уже на уровне базового исходника модель поддерживает как основные эксперименты, так и дополнительные задания без отдельного форка кода.

4.4 Базовый эксперимент

Базовый сценарий запускается скриптом `scripts/sir_run_basic.jl`, который формирует временные ряды `S(t)`, `I(t)`, `R(t)` и общей численности популяции.

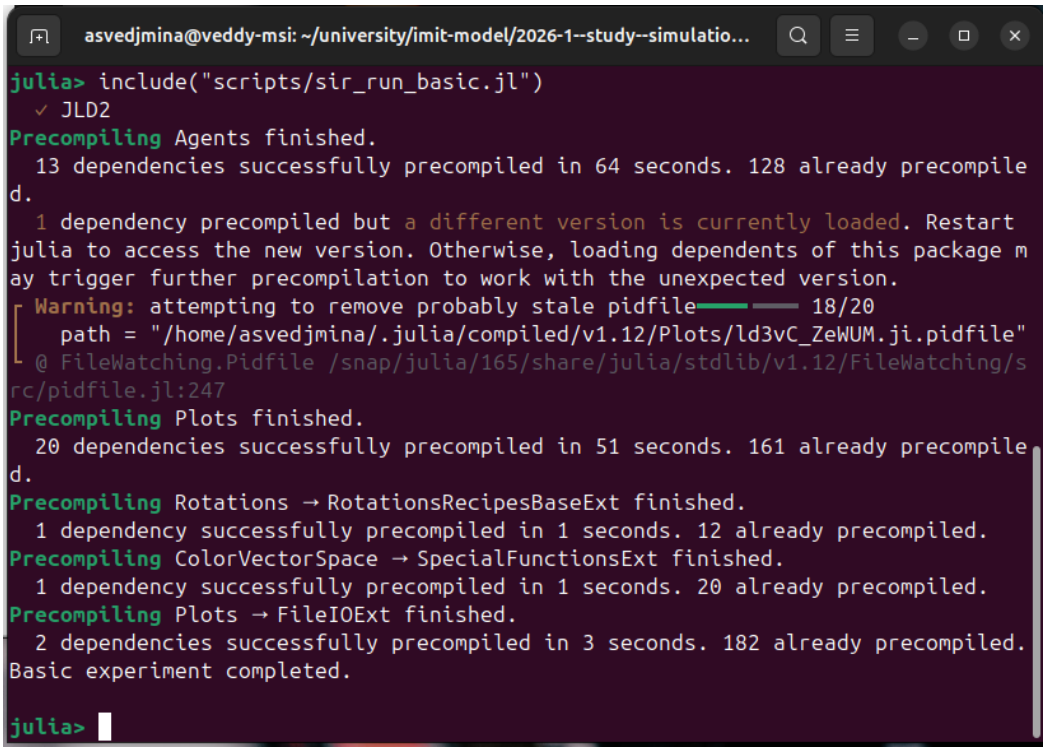

Рисунок 4.4: Запуск базового сценария

Результат базового эксперимента показан на рис. 4.5.

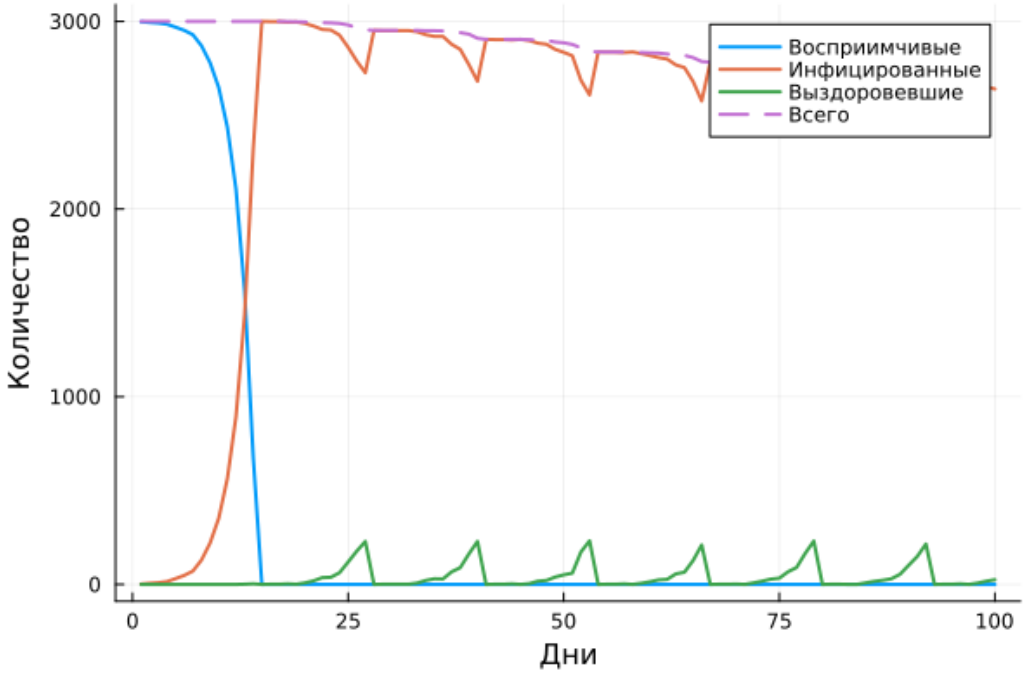

Рисунок 4.5: Базовая динамика агентной SIR-модели

Согласно `basic_summary.csv`, были получены следующие характеристики:

- теоретическое `R_0 = 7.0`;
- максимальное число инфицированных `3000`, то есть `100%` популяции;
- время достижения пика `15` дней;
- итоговое число умерших `335`.

Форма графика соответствует ожидаемому поведению при `R_0 \gg 1`: сначала число инфицированных быстро растёт, затем после достижения пика происходит спад за счёт перехода части агентов в класс `R` и за счёт смертности, а общая численность популяции уменьшается.

4.5 Исследование коэффициента заразности

Следующим шагом было выполнено сканирование коэффициента `beta` с помощью скрипта `scripts/sir_scan_beta.jl`.

```
julia> include("scripts/sir_scan_beta.jl")
  Activating project at `~/university/imit-model/2026-1--study--simulation-model
ing/labs/lab04/lab_04_models`
Precompiling Plots finished.
  2 dependencies successfully precompiled in 43 seconds. 179 already precompiled
.
Precompiling Plots → FileIOExt finished.
  1 dependency successfully precompiled in 2 seconds. 183 already precompiled.
Beta scan completed.

julia>
```

Рисунок 4.6: Запуск исследования коэффициента `beta`

Для каждого значения `beta` использовалось несколько значений `seed`, а результаты сохранялись в таблицу `beta_scan_all.csv`. Фрагмент этой таблицы показан на рис. 4.7.

beta_scan_all.csv — LibreOffice Calc

beta	seed	peak	final_inf	final_rec	deaths
0.1	42	0.000333333333333333	0	0.000333333333333333	0
0.1	43	0.002	0	0.00266666666666667	0
0.1	44	0.000333333333333333	0	0.000333333333333333	0
0.2	42	0.985	0.917666666666667	0.0383333333333333	132
0.2	43	0.001	0	0.001	0
0.2	44	0.000333333333333333	0	0.000333333333333333	0
0.3	42	0.998	0.884	0.00666666666666667	328
0.3	43	0.996666666666667	0.846666666666667	0.0546666666666667	296
0.3	44	0.000333333333333333	0	0.000333333333333333	0
0.4	42	1	0.81	0.0713333333333333	356
0.4	43	0.999	0.865333333333333	0.0266666666666667	324
0.4	44	0.000666666666666667	0	0.000666666666666667	0
0.5	42	1	0.88	0.00833333333333333	335
0.5	43	1	0.9	0.000333333333333333	299
0.5	44	1	0.893333333333333	0.000333333333333333	319
0.6	42	0.999666666666667	0.875333333333333	0.0126666666666667	336
0.6	43	1	0.885666666666667	0.00166666666666667	338
0.6	44	1	0.873333333333333	0.00333333333333333	370
0.7	42	1	0.783	0.0816666666666667	406
0.7	43	1	0.885	0.00233333333333333	338
0.7	44	1	0.872	0.0146666666666667	340
0.8	42	1	0.752	0.117333333333333	392
0.8	43	1	0.887666666666667	0.002	331
0.8	44	1	0.863666666666667	0.021	346
0.9	42	1	0.825333333333333	0.041	401
0.9	43	1	0.823666666666667	0.0546666666666667	365
0.9	44	1	0.869666666666667	0	391
1	42	1	0.869	0	393
1	43	1	0.865666666666667	0	403
1	44	1	0.858	0	426

Рисунок 4.7: Фрагмент файла `beta_scan_all.csv`

График усреднённых результатов представлен на рис. 4.8.

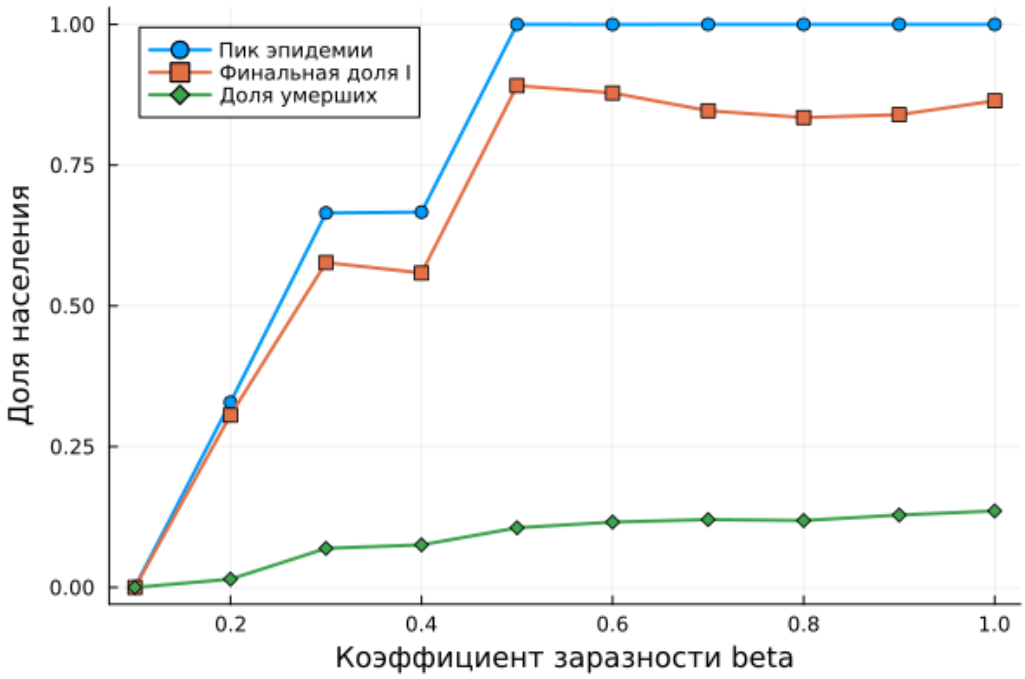

Рисунок 4.8: Сканирование коэффициента заразности

По графику и CSV-файлу видно, что:

- при `beta = 0.1` вспышка либо не возникает, либо остаётся очень слабой;
- около `beta = 0.2` система проходит через пороговый режим;
- при `beta >= 0.5` пик эпидемии захватывает почти всю популяцию;
- число умерших растёт вместе с усилением передачи.

Это исследование понадобилось далее для порогового анализа и для интерпретации результатов дополнительных заданий.

4.6 Исследование миграции

Влияние межгородских перемещений анализировалось отдельным сценарием миграционного анализа. Имя файла сценария: `sir_migration_effect.jl`.

```
julia> include("scripts/sir_migration_effect.jl")
Migration scan completed.

julia>
```

Рисунок 4.9: Запуск скрипта `sir_migration_effect.jl`

Результирующий график представлен на рис. 4.10.

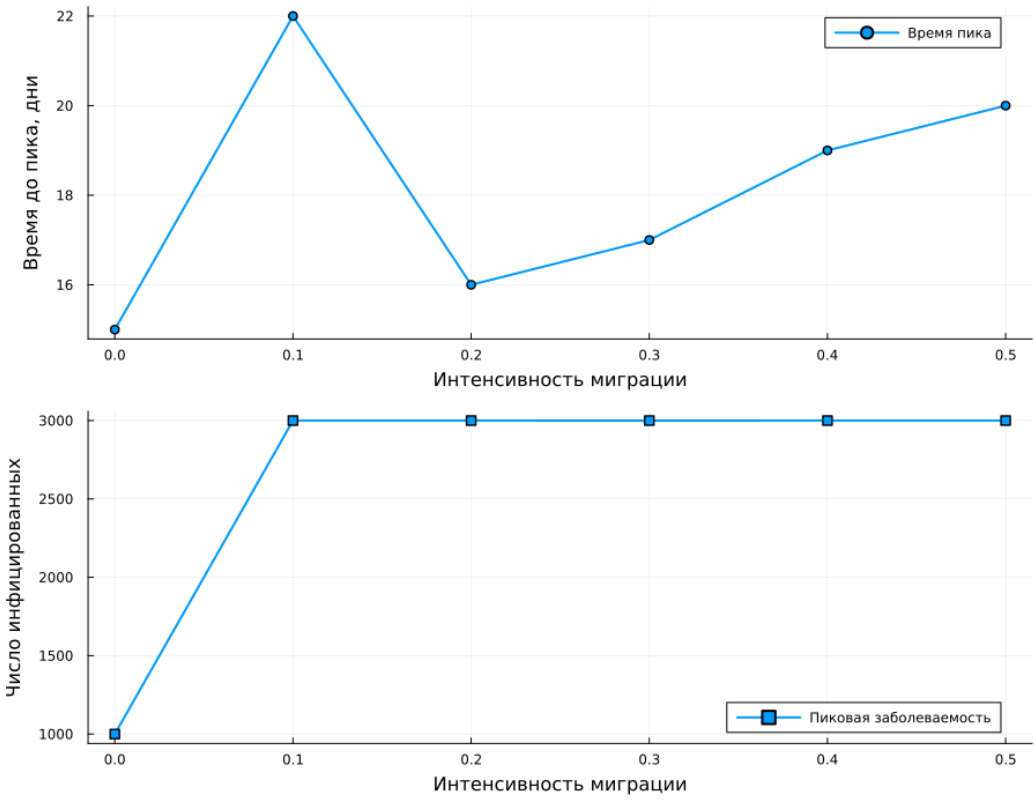

Рисунок 4.10: Влияние интенсивности миграции на время и масштаб пика

Из графика и таблицы `migration_scan_all.csv` следует:

- при нулевой миграции вспышка остаётся по существу локальной;
- при ненулевой миграции заражение переносится между городами и охватывает почти всю систему;
- среди миграционных режимов быстрое развитие эпидемии наблюдается уже при умеренных интенсивностях миграции.

Это подтверждает, что структура мобильности между городами является существенным фактором глобальной динамики.

4.7 Сценарные эксперименты

В `sir_analysis.jl` дополнительно реализована функция `scenario_parameters`, задающая четыре сценария:

- `baseline`;
- `heterogeneous_beta`;
- `fast_migration`;
- `slow_detection`.

Для каждого сценария строится отдельный график глобальной и по-городской динамики, после чего формируется сводная таблица `scenario_summary.csv`.

4.7.1 Базовый сценарий

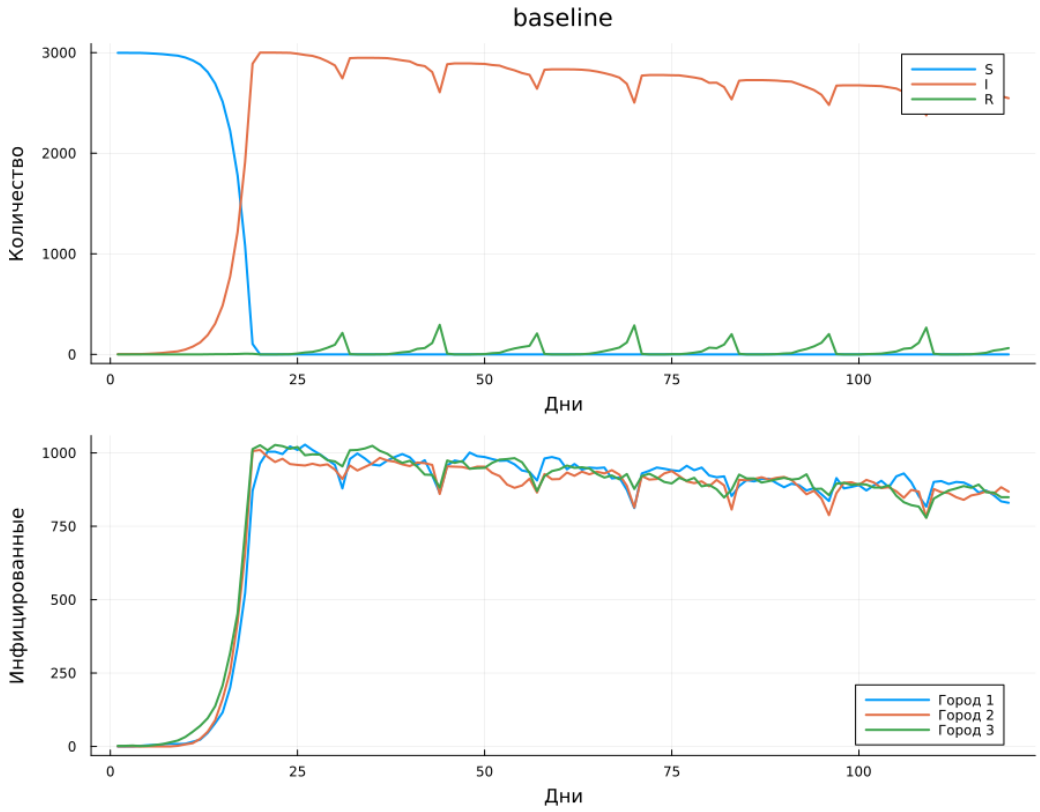

Рисунок 4.11: Сценарий `baseline`

Этот график используется как контрольная точка для всех остальных сценариев.

4.7.2 Неоднородная заразность

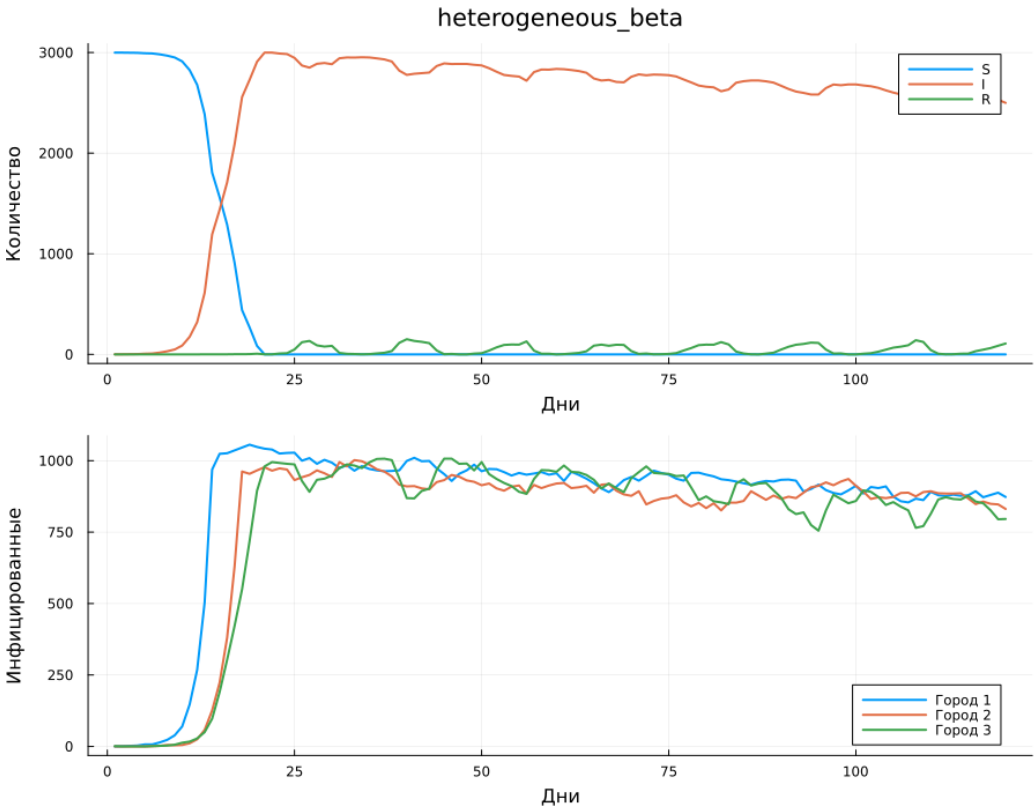

Рисунок 4.12: Сценарий `heterogeneous_beta`

В этом сценарии коэффициенты заразности по городам различаются, что приводит к асимметричному распределению локальных пиков.

4.7.3 Ускоренная миграция

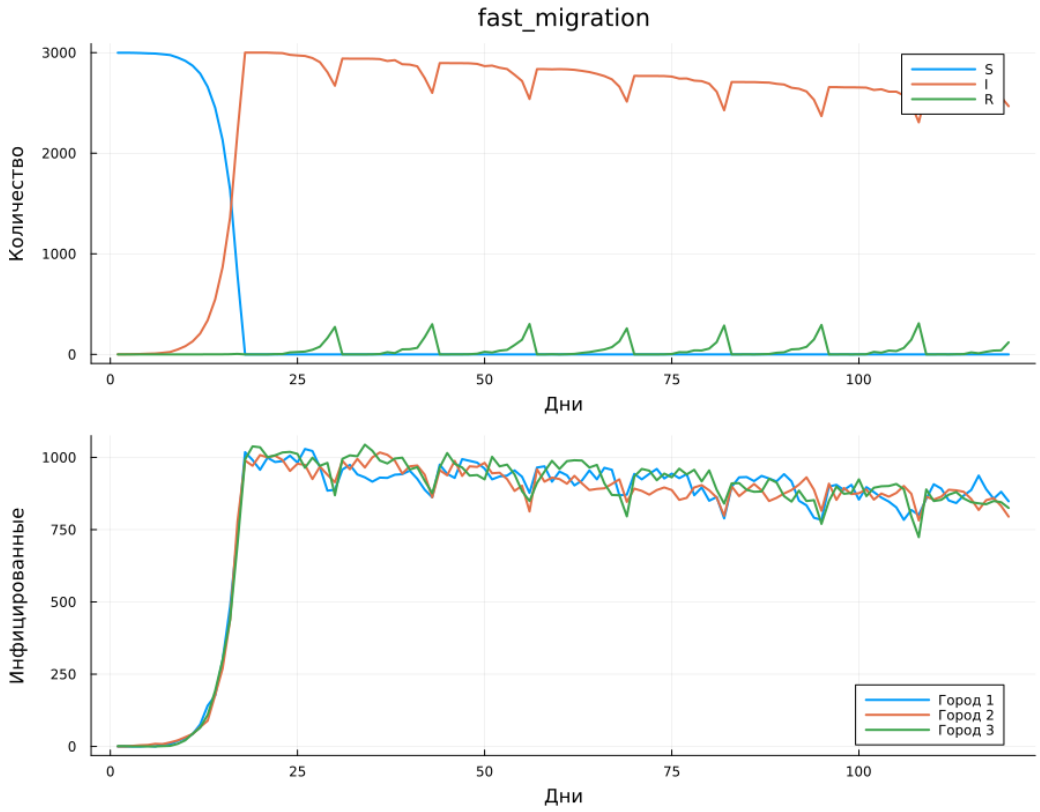

Рисунок 4.13: Сценарий `fast_migration`

Увеличение интенсивности миграции ускоряет межгородской перенос инфекции и делает вспышку более синхронной.

4.7.4 Позднее обнаружение инфекции

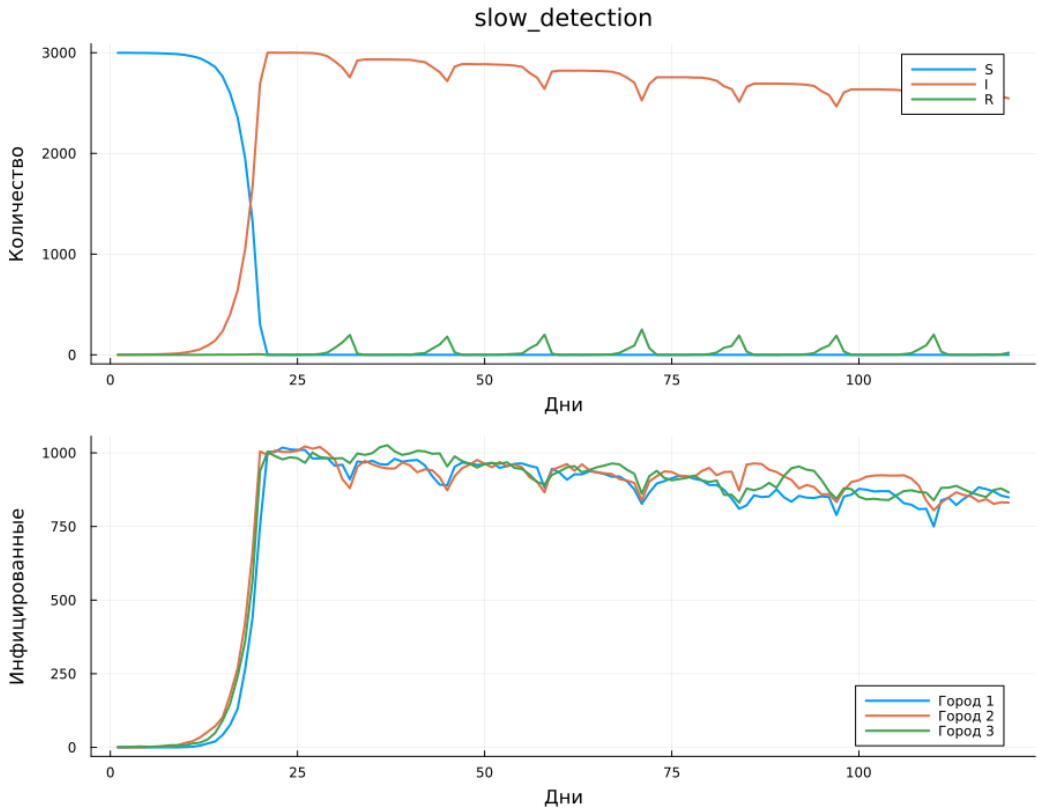

Рисунок 4.14: Сценарий `slow_detection`

Позднее переключение с `beta_und` на `beta_det` усиливает распространение болезни и увеличивает итоговую тяжесть эпидемии.

4.7.5 Сводное сравнение сценариев

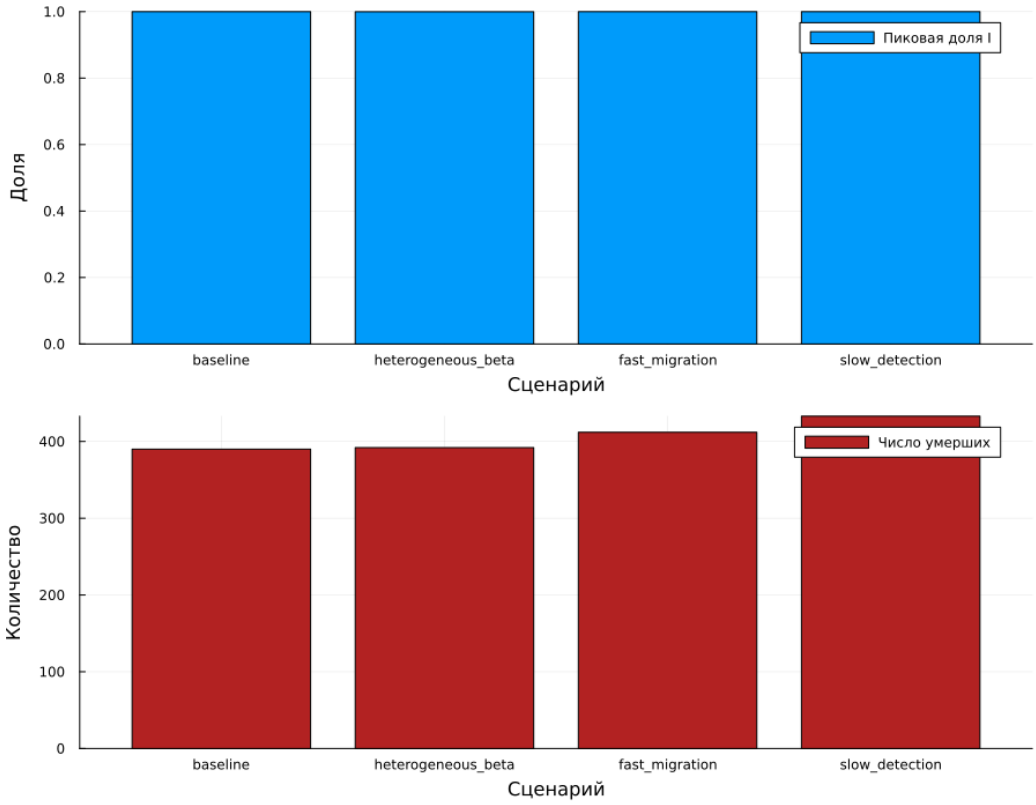

Рисунок 4.15: Сравнение сценариев по пику и смертности

Согласно `scenario_summary.csv`, наиболее тяжёлым из рассмотренных сценариев оказался `slow_detection`, где число умерших достигает 433, а в сценарии `fast_migration` эпидемия развивается быстрее, чем в базовом.

4.8 Оптимизация и итоговая комплексная визуализация

После основных сценариев был запущен скрипт построения сводных графиков `scripts/sir_visualize_dynamics.jl`.

```
julia> include("scripts/sir_visualize_dynamics.jl")
Comprehensive plot completed.

julia>
```

Рисунок 4.16: Запуск скрипта `sir_visualize_dynamics.jl`

Полученный многопанельный график приведён на рис. 4.17.

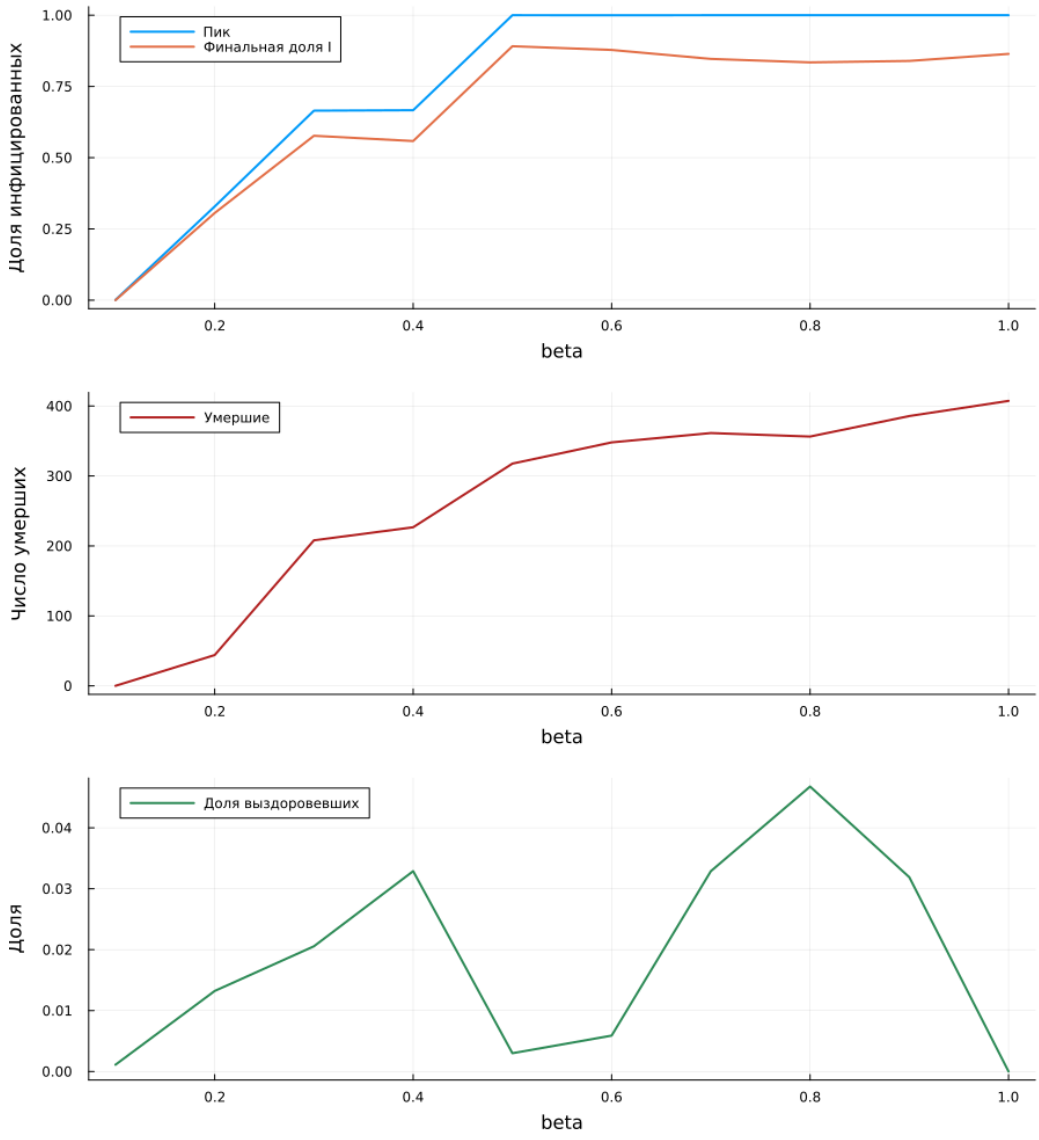

Рисунок 4.17: Комплексная визуализация зависимости метрик от `beta`

Он объединяет три семейства зависимостей:

- средний пик заражения;
- среднее число умерших;
- среднюю долю выздоровевших.

Такой график удобен тем, что сразу показывает компромисс между интенсивностью эпидемии, смертностью и финальной структурой популяции.

4.9 Литературное программирование и производные форматы

Главные эксперименты были оформлены в двух базовых literate-источниках:

- `sir_literate.jl`;
- `sir_param_literate.jl`.

Позднее к ним были добавлены отдельные literate-источники для дополнительных заданий:

- `sir_threshold_literate.jl`;
- `sir_heterogeneity_literate.jl`;
- `sir_quarantine_literate.jl`;
- `sir_optimize_literate.jl`.

Генерация производных форматов выполняется скриптом `generate.jl`.

```
asvedjmina@veddy-msi: ~/university/imit-model/2026-1--study--simulatio...
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab04/lab_04_models$ julia --project=. scripts/generate.jl
Precompiling DrWatson finished.
  3 dependencies successfully precompiled in 21 seconds. 42 already precompiled.
Precompiling Literate finished.
  2 dependencies successfully precompiled in 3 seconds. 17 already precompiled.
[ Info: generating plain script file from `~/university/imit-model/2026-1--study
--simulation-modeling/labs/lab04/lab_04_models/scripts/sir_literate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mod
eling/labs/lab04/lab_04_models/scripts/sir_clean.jl`
[ Info: generating notebook from `~/university/imit-model/2026-1--study--simulat
ion-modeling/labs/lab04/lab_04_models/scripts/sir_literate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mod
eling/labs/lab04/lab_04_models/notebooks/sir.ipynb`
[ Info: generating markdown page from `~/university/imit-model/2026-1--study--si
mulation-modeling/labs/lab04/lab_04_models/scripts/sir_literate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mod
eling/labs/lab04/lab_04_models/docs/sir.qmd`
[ Info: generating plain script file from `~/university/imit-model/2026-1--study
--simulation-modeling/labs/lab04/lab_04_models/scripts/sir_param_literate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mod
eling/labs/lab04/lab_04_models/scripts/sir_param_clean.jl`
[ Info: generating notebook from `~/university/imit-model/2026-1--study--simulat
ion-modeling/labs/lab04/lab_04_models/scripts/sir_param_literate.jl`
```

Рисунок 4.18: Генерация производных форматов через `generate.jl`

```
asvedjmina@veddy-msi: ~/university/imit-model/2026-1--study--simulation...
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/l
abs/lab04/lab_04_models$ julia --project=. scripts/generate.jl
Precompiling Literate finished.
  2 dependencies successfully precompiled in 4 seconds. 17 already precompiled.
[ Info: generating plain script file from `~/university/imit-model/2026-1--study-
-simulation-modeling/labs/lab04/lab_04_models/scripts/sir_literate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mode
ling/labs/lab04/lab_04_models/scripts/sir_clean.jl`
[ Info: generating notebook from `~/university/imit-model/2026-1--study--simulati
on-modeling/labs/lab04/lab_04_models/scripts/sir_literate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mode
ling/labs/lab04/lab_04_models/notebooks/sir.ipynb`
[ Info: generating markdown page from `~/university/imit-model/2026-1--study--sim
ulation-modeling/labs/lab04/lab_04_models/scripts/sir_literate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mode
ling/labs/lab04/lab_04_models/docs/sir.qmd`
[ Info: generating plain script file from `~/university/imit-model/2026-1--study-
-simulation-modeling/labs/lab04/lab_04_models/scripts/sir_param_literate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mode
ling/labs/lab04/lab_04_models/scripts/sir_param_clean.jl`
[ Info: generating notebook from `~/university/imit-model/2026-1--study--simulati
on-modeling/labs/lab04/lab_04_models/scripts/sir_param_literate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mode
ling/labs/lab04/lab_04_models/notebooks/sir_param.ipynb`
[ Info: generating markdown page from `~/university/imit-model/2026-1--study--sim
ulation-modeling/labs/lab04/lab_04_models/scripts/sir_param_literate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mode
ling/labs/lab04/lab_04_models/docs/sir_param.qmd`
Literate generation completed.
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/l
abs/lab04/lab_04_models$
```

Рисунок 4.19: Повторная генерация literate-артефактов после расширения проекта

Сгенерированные notebook располагаются в каталоге `notebooks/`, Quarto-файлы — в `docs/`, а чистые Julia-скрипты — в `scripts/`.

Итоговый набор производных файлов имеет вид:

- `sir_literate.jl`: clean `sir_clean.jl`, notebook `sir.ipynb`, quarto `sir.qmd`.
- `sir_param_literate.jl`: clean `sir_param_clean.jl`, notebook `sir_param.ipynb`, quarto `sir_param.qmd`.
- `sir_threshold_literate.jl`: clean `sir_threshold_clean.jl`, notebook `sir_threshold.ipynb`, quarto `sir_threshold.qmd`.
- `sir_heterogeneity_literate.jl`: clean `sir_heterogeneity_clean.jl`, notebook `sir_heterogeneity.ipynb`, quarto `sir_heterogeneity.qmd`.

- `sir_quarantine_literate.jl`: clean `sir_quarantine_clean.jl`, notebook `sir_quarantine.ipynb`, quarto `sir_quarantine.qmd`.
- `sir_optimize_literate.jl`: clean `sir_optimize_clean.jl`, notebook `sir_optimize.ipynb`, quarto `sir_optimize.qmd`.

Использование единого literate-источника для каждого эксперимента устраняет рассинхронизацию между кодом, notebook и документацией.

5 Дополнительные задания

5.1 Задание 1. Базовый анализ R_0

В базовом режиме было получено:

- `R_0 = 7.0`;
- `peak_fraction = 1.0`;
- `peak_time = 15`;
- `deaths = 335`.

Следовательно, уже по первому дополнительному заданию видно, что модель находится глубоко в надпороговом режиме: инфекция уверенно распространяется по всей популяции, а наблюдаемая динамика полностью согласуется с теоретической оценкой `R_0`.

5.2 Задание 2. Порог эпидемии

Для поиска практического эпидемического порога был реализован отдельный скрипт `sir_threshold_study.jl`.

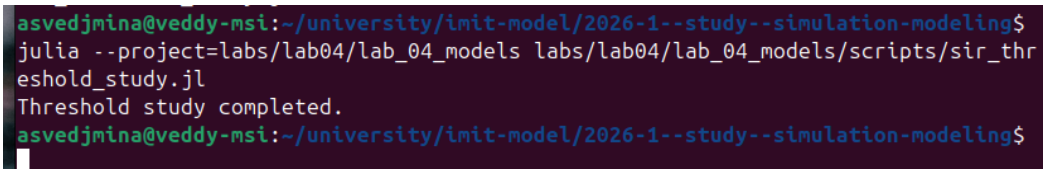

Рисунок 5.1: Запуск порогового исследования

График порогового исследования приведён на рис. 5.2.

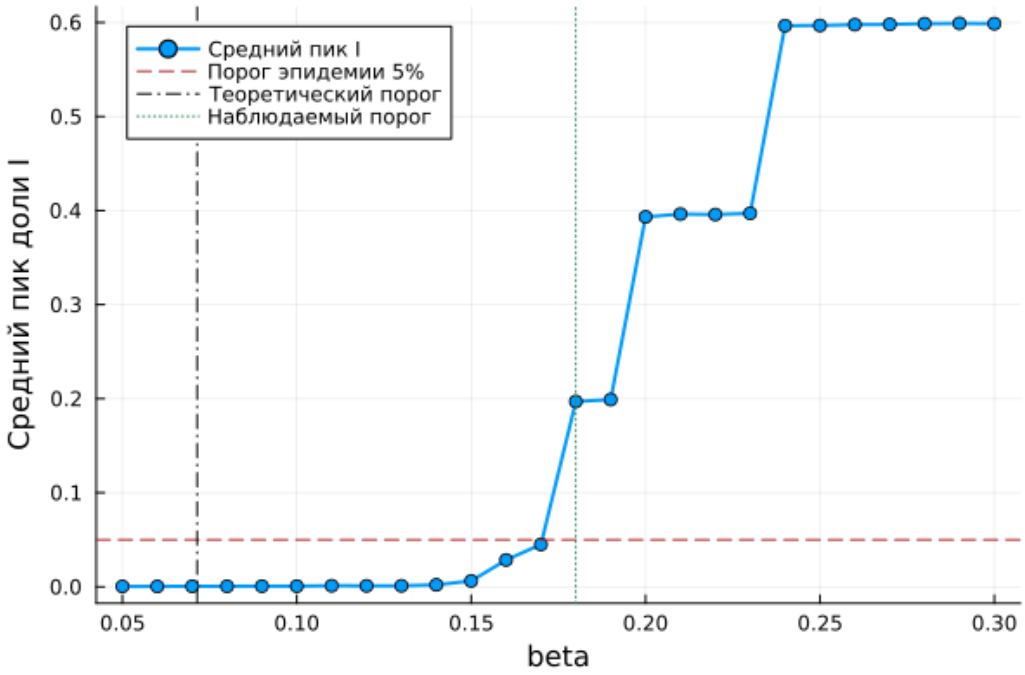

Рисунок 5.2: Пороговое исследование по коэффициенту `beta`

По таблице `threshold_summary.csv` были получены значения:

- эмпирический критерий эпидемии: `peak > 5%`;
- наблюдаемый порог: `beta ≈ 0.18`;
- теоретический порог: `beta ≈ 0.0714`;
- разрыв между ними: `≈ 0.1086`.

Такой разрыв естественен для агентной стохастической модели: теоретический порог соответствует только условию `R_0 = 1`, а практический критерий требует заметной и устойчивой вспышки.

5.3 Задание 3. Эффект гетерогенности

Отдельное исследование неоднородной заразности выполнялось скриптом `sir_heterogeneity_effect.jl`.

```
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling$
julia --project=labs/lab04/lab_04_models labs/lab04/lab_04_models/scripts/sir_hetero
geneity_effect.jl
Heterogeneity experiment completed.
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling$
```

Рисунок 5.3: Запуск сценария гетерогенности

Для анализа были построены три графика.

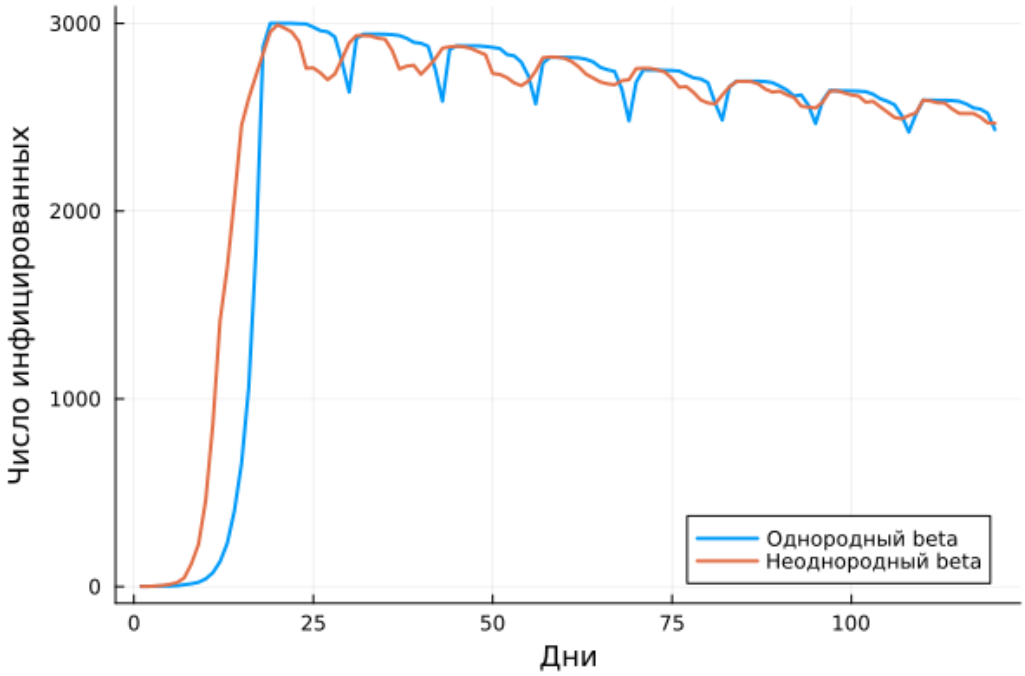

Рисунок 5.4: Сравнение глобальной динамики для однородного и неоднородного сценариев

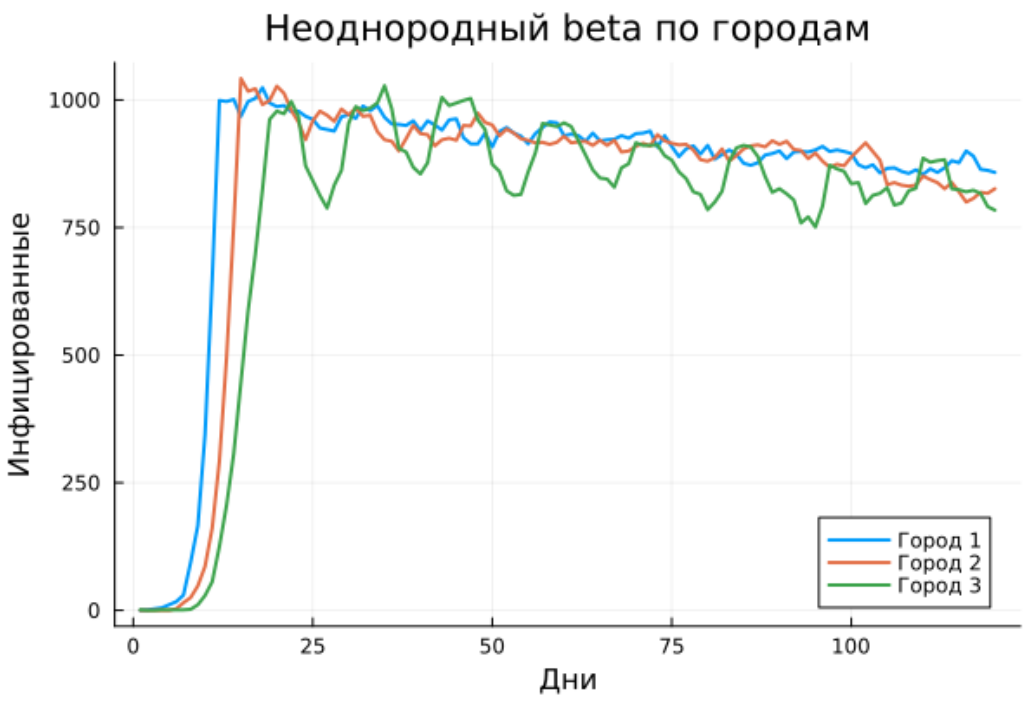

Рисунок 5.5: Покомпонентная динамика заражённых по городам

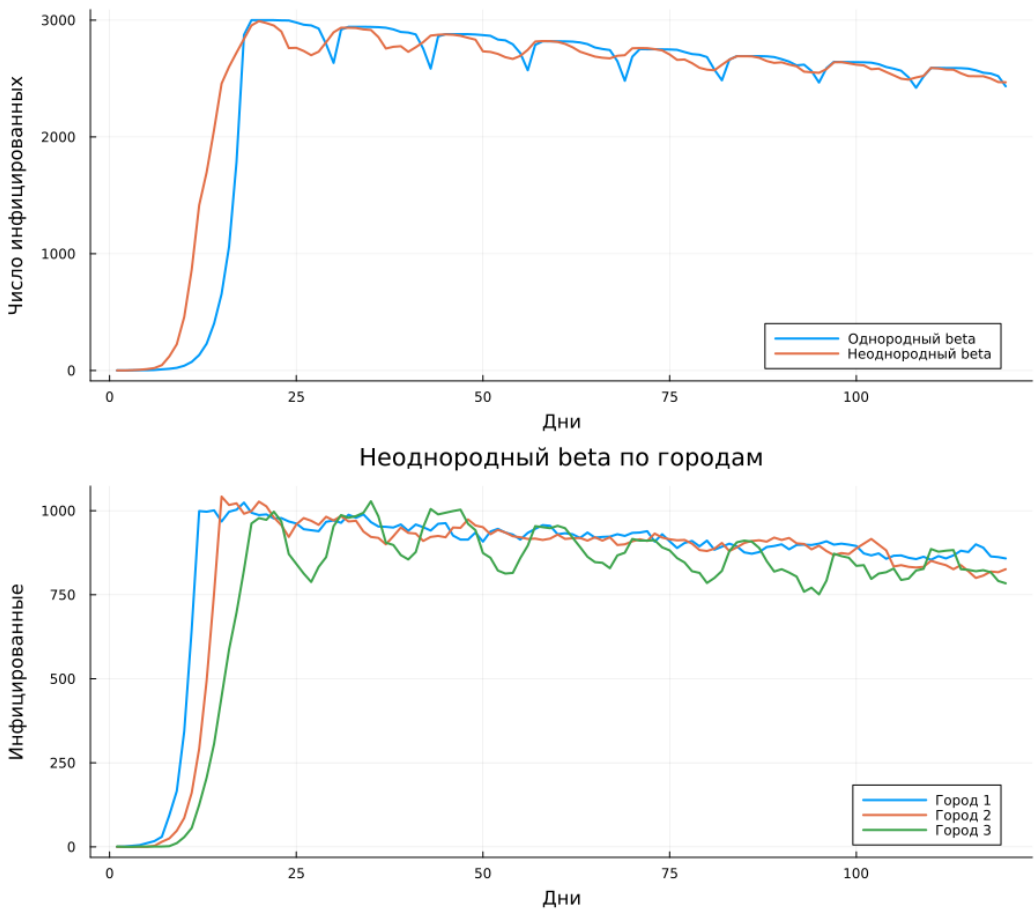

Рисунок 5.6: Сводная визуализация эффекта гетерогенности

По `heterogeneity_summary.csv` и `heterogeneity_city_summary.csv` получаем:

- однородный режим: пик `3000`, смертность `429`;
- неоднородный режим: пик `2992`, смертность `444`;
- пиковые значения по городам при `beta = [0.8, 0.45, 0.2]` составили `1024`, `1042`, `1028`.

Иными словами, неоднородность не устраняет эпидемию, а перераспределяет её локальную интенсивность между городами и слегка ухудшает итог по смертности.

5.4 Задание 4. Влияние миграции

Это задание содержательно опирается на уже выполненный выше миграционный эксперимент. Дополнительный вывод состоит в том, что для межгородского распространения эпидемии важна не только величина пика, но и скорость его достижения.

```
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling$
julia --project=labs/lab04/lab_04_models labs/lab04/lab_04_models/scripts/sir_mig
ration_effect.jl
Migration scan completed.
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling$
```

Рисунок 5.7: Запуск дополнительного анализа миграции

График на рис. 4.10 показывает, что уже при умеренной миграции вспышка становится общей для всех городов, а время до глобального пика уменьшается по сравнению с разрозненными локальными очагами.

5.5 Задание 5. Карантинные меры

Для анализа карантина был реализован скрипт `sir_quarantine.jl`.

```
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling$
julia --project=labs/lab04/lab_04_models labs/lab04/lab_04_models/scripts/sir_qua
rantine.jl
Quarantine experiment completed.
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling$
```

Рисунок 5.8: Запуск карантинного сценария

В эксперименте сравнивались сценарий без карантина и несколько значений порога закрытия города.

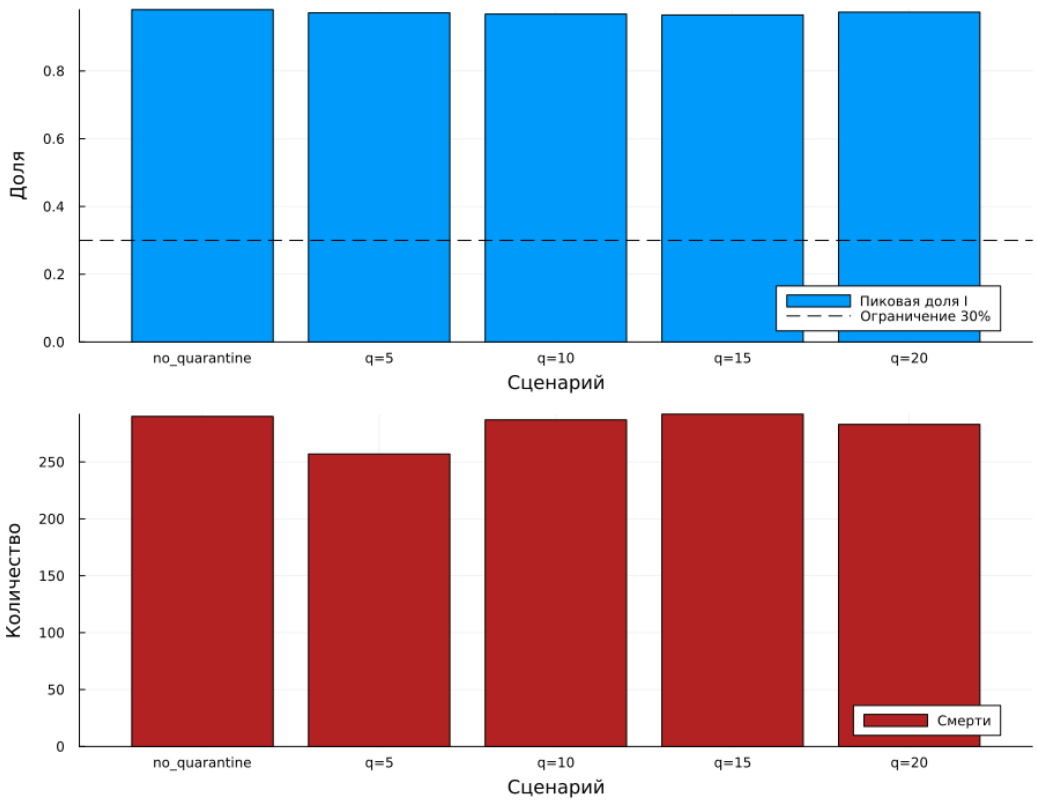

Рисунок 5.9: Сравнение карантинных режимов по пику и смертности

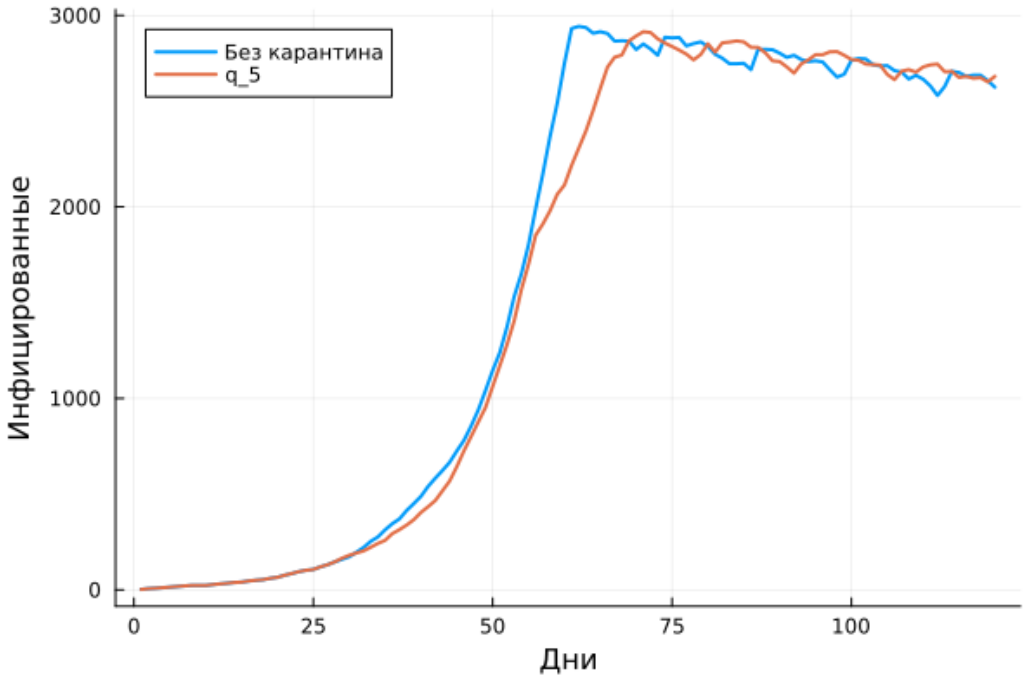

Рисунок 5.10: Сравнение лучшего карантинного режима с базовым по временной динамике

Итоговая сводка `quarantine_summary.csv` показывает:

Сценарий	Порог	Пиковая доля I	Время пика	Смерти	Закрытые города
no_quarantine	-1	0.9810	62	290	0
q_5	0.05	0.9717	71	257	3
q_10	0.10	0.9680	66	287	3
q_15	0.15	0.9653	64	292	3
q_20	0.20	0.9737	65	283	3

Наиболее удачным из проверенных оказался порог 5%: он уменьшает число смертей и одновременно оттягивает пик эпидемии.

5.6 Задание 6. Оптимизация при ограничении на пик

Последнее дополнительное задание решалось отдельным сценарием оптимизации. Имя файла сценария: `sir_optimize_parameters.jl`.

```
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling$
julia --project=labs/lab04/lab_04_models labs/lab04/lab_04_models/scripts/sir_opt
imize_parameters.jl
Optimization completed.
Best feasible candidate:
  beta_und            = 0.05
  detection_time      = 3
  migration_intensity = 0.0
  mean_peak           = 0.0003
  mean_deaths         = 0.0
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling$
```

Рисунок 5.11: Запуск сценария оптимизации

Для поиска решений использован многокритериальный эволюционный алгоритм `Borg MOEA` из пакета `BlackBoxOptim`. Оптимизируемые параметры: `beta_und`, `detection_time` и `death_rate`. Алгоритм строит Pareto-фронт по двум критериям: средний пик заболеваемости и среднее число умерших. После этого из фронта выбираются допустимые решения с ограничением `mean_peak <= 0.30`.

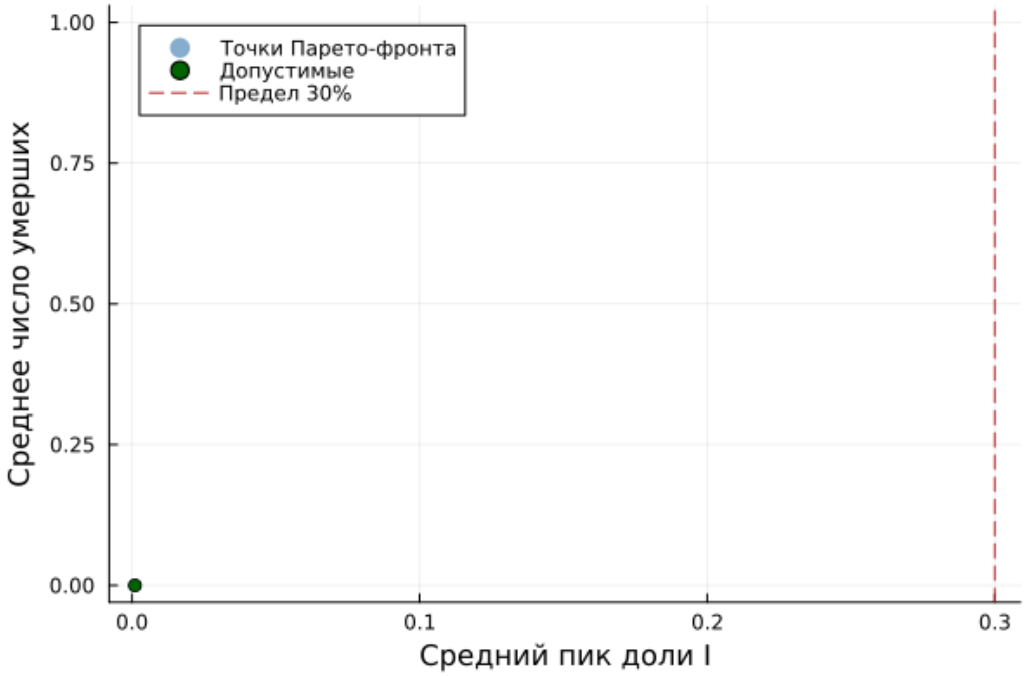

Рисунок 5.12: Фронт допустимых решений при ограничении на пик

Лучшая допустимая комбинация по `optimization_pareto.csv`:

- `beta_und = 0.1409`;
- `detection_time = 4`;
- `death_rate = 0.0497`;
- `mean_peak = 0.0010`;
- `mean_deaths = 0.0`.

Эта картина показывает, что даже при ненулевой смертности и относительно небольшом, но не экстремально малом `beta`, раннее выявление удерживает эпидемию глубоко ниже заданного порога по пику.

5.7 Выполнение Jupyter notebook

После генерации производных форматов Jupyter notebook из каталога `notebooks/` были выполнены и сохранены уже с результатами.

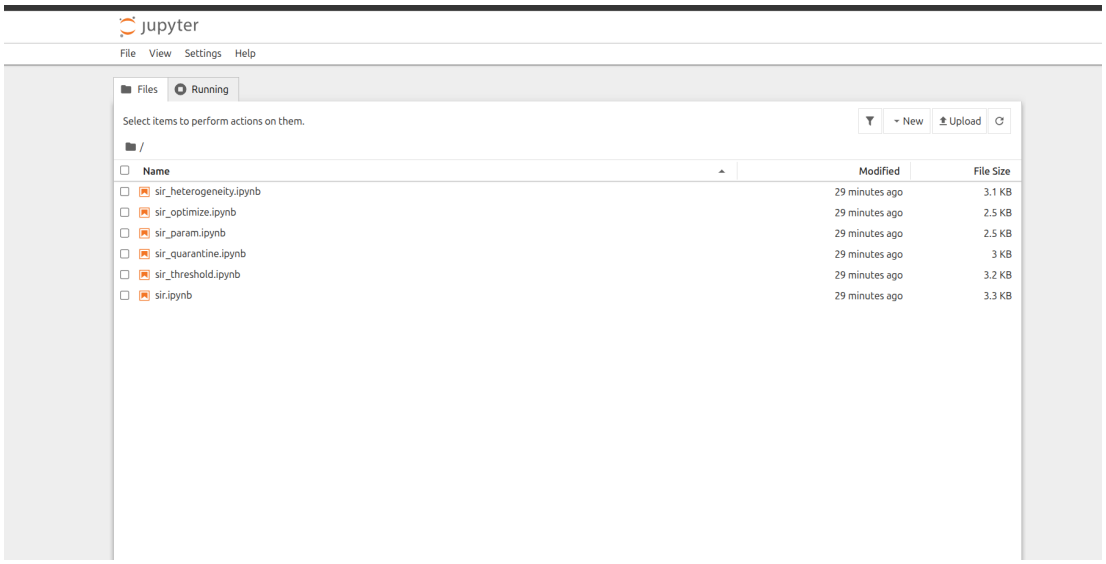

Рисунок 5.13: Открытая папка `notebooks/` в Jupyter

В репозитории сохранены все шесть выполненных notebook:

- `sir.ipynb`;
- `sir_param.ipynb`;
- `sir_threshold.ipynb`;
- `sir_heterogeneity.ipynb`;
- `sir_quarantine.ipynb`;
- `sir_optimize.ipynb`.

Ниже приведены скриншоты соответствующих блокнотов:

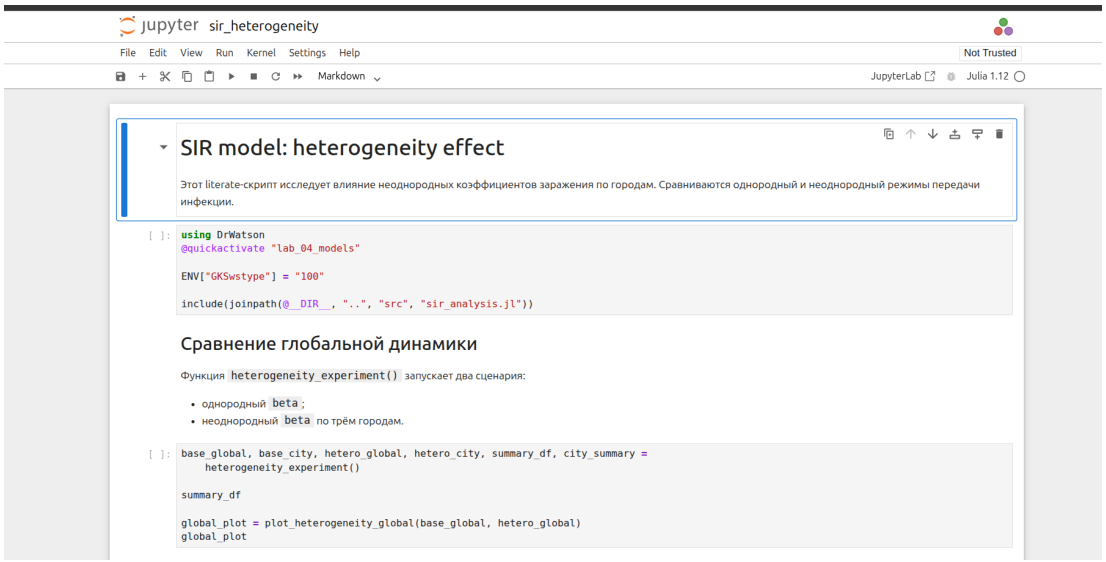

Рисунок 5.14: Notebook `sir_heterogeneity.ipynb`

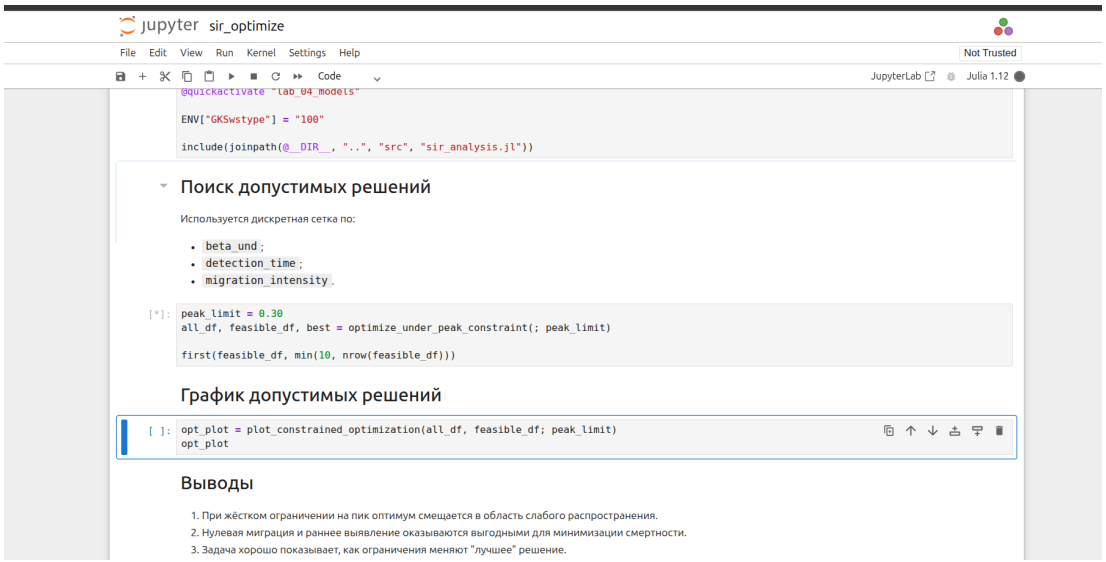

Рисунок 5.15: Notebook `sir_optimize.ipynb`

jupyter sir_param

File Edit View Run Kernel Settings Help Not Trusted

Code JupyterLab Julia 1.12

Parameter scenarios for the graph SIR model

This script extends the base experiment with several parameter scenarios: heterogeneous transmission, faster migration, and slower detection.

Libraries and project activation

[*]:
```
using DrWatson
@quickactivate "lab_04_models"

ENV["GKSwstype"] = "100"

include(joinpath(@__DIR__, "..", "src", "sir_analysis.jl"))
```

Scenario list

Each scenario modifies a small set of parameters while keeping the model structure unchanged.

[*]:
```
scenarios = scenario_parameters()
summaries = NamedTuple[]

for spec in scenarios
    global_df, city_df, summary = run_scenario(spec)
    scenario_plot = plot_scenario(global_df, city_df; title = spec[:label])
    scenario_plot
    plot_name = "scenario_" * spec[:label] * ".png"
```

Рисунок 5.16: Notebook `sir_param.ipynb`

jupyter sir_quarantine

File Edit View Run Kernel Settings Help Trusted

Markdown JupyterLab Julia 1.12

SIR model: quarantine policy

Этот literate-скрипт исследует карантинную меру: город закрывается, если доля инфицированных в нём превышает заданный порог.

[*]:
```
using DrWatson
@quickactivate "lab_04_models"

ENV["GKSwstype"] = "100"

include(joinpath(@__DIR__, "..", "src", "sir_analysis.jl"))
```

Серия карантинных порогов

Сценарий сравнивает режим без карантина с несколькими значениями `quarantine_threshold`.

[*]:
```
summary_df, dynamics = quarantine_experiment()
summary_df

summary_plot = plot_quarantine_summary(summary_df)
summary_plot
```

Сравнение лучшего карантинного режима с базовым

Из всех карантинных сценариев выбирается режим с минимальным числом смертей.

[*]:
```
candidate_df = filter(row -> row.scenario != "no_quarantine" && row.locked_cities > 0, summary_df)
```

Рисунок 5.17: Notebook `sir_quarantine.ipynb`

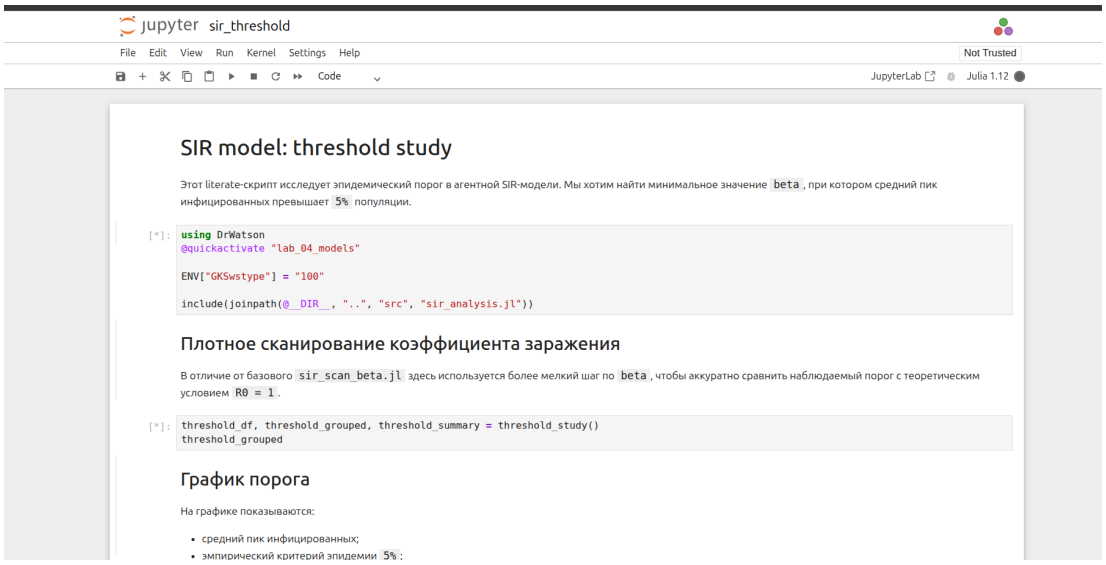

Рисунок 5.18: Notebook `sir_threshold.ipynb`

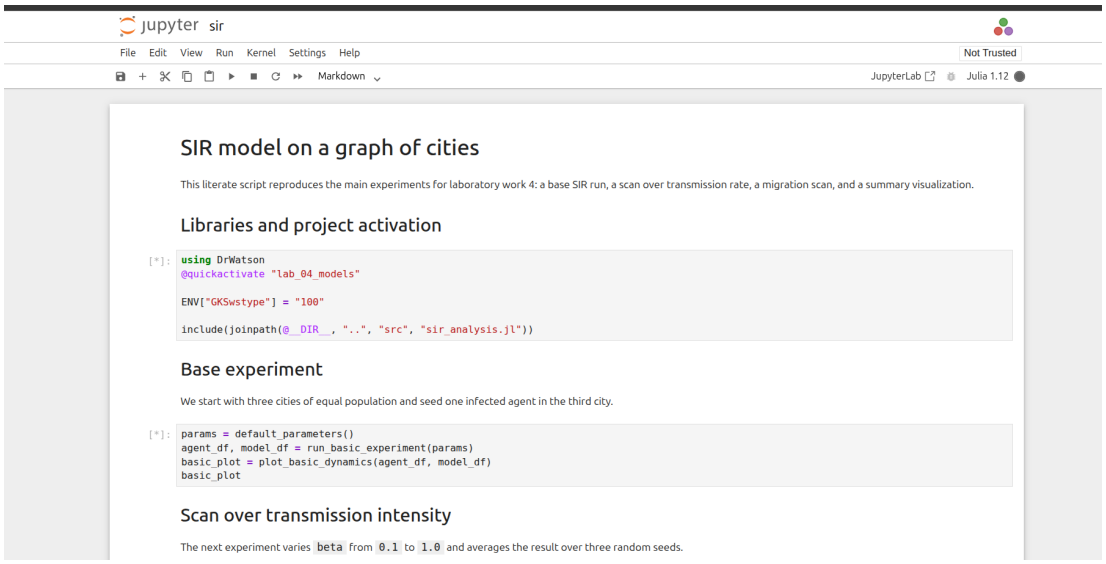

Рисунок 5.19: Notebook `sir.ipynb`

Скриншоты показывают работу с notebook в Jupyter, а сами файлы `.ipynb` сохранены с заполненными `execution_count` и блоками `outputs`, то есть подтверждают именно выполнение, а не только генерацию.

6 Полный код проекта

В приложении приводится полный авторский код проекта:

- исходные модули из `src/`;
- исполняемые сценарии из `scripts/`;
- literate-источники;
- генератор производных форматов;
- тесты.

Автоматически сгенерированные `*_clean.jl`, `.ipynb` и `.qmd` не дублируются в тексте отчёта, поскольку они полностью воспроизводятся из literate-источников скриптом `generate.jl`.

6.1 Исходный код модели `sir_model.jl`

```
using Agents
using Agents.Graphs
using Distributions: Poisson
using DrWatson: @dict
using Random
using StatsBase: Weights, sample

@agent struct Person(GraphAgent)
    days_infected::Int
    status::Symbol
    home_city::Int
end

function default_migration_matrix(Ns)
    C = length(Ns)
    migration_rates = zeros(Float64, C, C)
```

```
    for i in 1:C
        for j in 1:C
            migration_rates[i, j] = (Ns[i] + Ns[j]) / Ns[i]
        end
        migration_rates[i, :] ./= sum(migration_rates[i, :])
    end
    return migration_rates
end

function create_migration_matrix(C::Integer, intensity::Real)
    @assert C >= 2 "At least two cities are required"
    @assert 0.0 <= intensity <= 1.0 "Migration intensity must be in [0, 1]"

    migration_rates = fill(intensity / (C - 1), C, C)
    for i in 1:C
        migration_rates[i, i] = 1 - intensity
    end
    return migration_rates
end

function initialize_sir(;
    Ns = [1000, 1000, 1000],
    migration_rates = nothing,
    beta_und = [0.5, 0.5, 0.5],
    beta_det = [0.05, 0.05, 0.05],
    infection_period = 14,
    detection_time = 7,
    death_rate = 0.02,
    reinfection_probability = 0.1,
    Is = [0, 0, 1],
    seed = 42,
    n_steps = 100,
    quarantine_threshold = nothing,
)
    C = length(Ns)
    @assert C == length(beta_und) == length(beta_det) == length(Is) "Parameter lengths must match"

    rng = Xoshiro(seed)
    migration_rates === nothing && (migration_rates = default_migration_matrix(Ns))
    migration_rates = Float64.(migration_rates)
    city_locked_down = falses(C)

    properties = @dict(
        Ns,
        beta_und,
        beta_det,
        migration_rates,
        infection_period,
        detection_time,
        death_rate,
        reinfection_probability,
        C,
        n_steps,
        quarantine_threshold,
        city_locked_down,
    )

    space = GraphSpace(complete_graph(C))
```

```
    model = StandardABM(Person, space; properties, rng, agent_step! = sir_agent_step!)

    for city in 1:C
        for _ in 1:Ns[city]
            add_agent!(city, model, 0, :S, city)
        end
    end

    for city in 1:C
        Is[city] == 0 && continue
        city_agents = ids_in_position(city, model)
        infected_ids = sample(rng, city_agents, Is[city]; replace = false)
        for id in infected_ids
            agent = model[id]
            agent.status = :I
            agent.days_infected = 1
        end
    end

    return model
end

function sir_agent_step!(agent, model)
    migrate!(agent, model)
    agent.status == :I && transmit!(agent, model)
    agent.status == :I && (agent.days_infected += 1)
    recover_or_die!(agent, model)
    return nothing
end

function migrate!(agent, model)
    current_city = agent.pos
    probs = model.migration_rates[current_city, :]
    target_city = sample(abmrng(model), 1:model.C, Weights(probs))
    target_city == current_city && return nothing
    move_agent!(agent, target_city, model)
    return nothing
end

function transmit!(agent, model)
    rate = if agent.days_infected < model.detection_time
        model.beta_und[agent.pos]
    else
        model.beta_det[agent.pos]
    end

    n_infections = rand(abmrng(model), Poisson(rate))
    n_infections == 0 && return nothing

    neighbors = [a for a in agents_in_position(agent.pos, model) if a.id != agent.id]
    shuffle!(abmrng(model), neighbors)

    for contact in neighbors
        if contact.status == :S
            contact.status = :I
            contact.days_infected = 1
            n_infections -= 1
        elseif contact.status == :R && rand(abmrng(model)) <= model.reinfection_probability
```

```
            contact.status = :I
            contact.days_infected = 1
            n_infections -= 1
        end

        n_infections == 0 && return nothing
    end

    return nothing
end

function recover_or_die!(agent, model)
    if agent.status == :I && agent.days_infected >= model.infection_period
        if rand(abmrng(model)) <= model.death_rate
            remove_agent!(agent, model)
        else
            agent.status = :R
            agent.days_infected = 0
        end
    end
    return nothing
end

function safe_step!(model, n::Int = 1)
    for _ in 1:n
        for id in collect(allids(model))
            agent = try
                model[id]
            catch
                nothing
            end
            agent === nothing || sir_agent_step!(agent, model)
        end
        update_quarantine!(model)
    end
    return model
end

infected_count(model) = count(a.status == :I for a in allagents(model))
recovered_count(model) = count(a.status == :R for a in allagents(model))
susceptible_count(model) = count(a.status == :S for a in allagents(model))
total_count(model) = nagents(model)
locked_city_count(model) = count(identity, model.city_locked_down)

function city_status_count(model, city::Integer, status::Symbol)
    return count(a.status == status for a in agents_in_position(city, model))
end

city_population(model, city::Integer) = length(collect(ids_in_position(city, model)))

function infected_fraction(model; denominator = sum(model.Ns))
    denominator == 0 && return 0.0
    return infected_count(model) / denominator
end

function lock_city!(city::Integer, model)
    model.city_locked_down[city] && return nothing
    model.city_locked_down[city] = true
```

```
    model.migration_rates[city, :] .= 0.0
    model.migration_rates[city, city] = 1.0
    return nothing
end

function update_quarantine!(model)
    threshold = model.quarantine_threshold
    threshold === nothing && return nothing

    for city in 1:model.C
        model.city_locked_down[city] && continue
        population = city_population(model, city)
        population == 0 && continue
        infected_share = city_status_count(model, city, :I) / population
        infected_share >= threshold && lock_city!(city, model)
    end

    return nothing
end
```

6.2 Исходный код аналитики sir_analysis.jl

```
include("sir_model.jl")

using BlackBoxOptim
using CSV
using DataFrames
using JLD2
using Plots
using Random
using Statistics

function default_parameters()
    return Dict(
        :Ns => [1000, 1000, 1000],
        :beta_und => [0.5, 0.5, 0.5],
        :beta_det => [0.05, 0.05, 0.05],
        :infection_period => 14,
        :detection_time => 7,
        :death_rate => 0.02,
        :reinfection_probability => 0.1,
        :Is => [0, 0, 1],
        :seed => 42,
        :n_steps => 100,
    )
end

function collect_time_series(model, n_steps::Int)
    times = Int[]
    susceptible = Int[]
    infected = Int[]
    recovered = Int[]
    total = Int[]
```

```
    city_infected = [Int[] for _ in 1:model.C]

    for step in 1:n_steps
        safe_step!(model)
        push!(times, step)
        push!(susceptible, susceptible_count(model))
        push!(infected, infected_count(model))
        push!(recovered, recovered_count(model))
        push!(total, total_count(model))
        for city in 1:model.C
            push!(city_infected[city], city_status_count(model, city, :I))
        end
    end

    global_df = DataFrame(
        time = times,
        susceptible = susceptible,
        infected = infected,
        recovered = recovered,
        total = total,
    )
    city_df = DataFrame(time = times)
    for city in 1:model.C
        city_df[!, Symbol("city_$(city)_infected")] = city_infected[city]
    end

    return global_df, city_df
end

function run_basic_experiment(params::AbstractDict = default_parameters())
    model = initialize_sir(; pairs(params)...)
    global_df, _ = collect_time_series(model, params[:n_steps])
    agent_df = select(global_df, :time, :susceptible, :infected, :recovered)
    model_df = select(global_df, :time, :total)
    return agent_df, model_df
end

function plot_basic_dynamics(agent_df::DataFrame, model_df::DataFrame)
    plt = plot(
        agent_df.time,
        agent_df.susceptible;
        label = "Восприимчивые",
        xlabel = "Дни",
        ylabel = "Количество",
        linewidth = 2,
    )
    plot!(plt, agent_df.time, agent_df.infected; label = "Инфицированные", linewidth = 2)
    plot!(plt, agent_df.time, agent_df.recovered; label = "Выздоровевшие", linewidth = 2)
    plot!(plt, model_df.time, model_df.total; label = "Всего", linestyle = :dash, linewidth = 2)
    return plt
end

reproduction_number(params::AbstractDict) = first(params[:beta_und]) * params[:infection_period]

function basic_summary(params::AbstractDict = default_parameters())
    model = initialize_sir(; pairs(params)...)
    global_df, _ = collect_time_series(model, params[:n_steps])
    initial_population = sum(params[:Ns])
```

```
    peak_idx = argmax(global_df.infected)
    peak_infected = global_df.infected[peak_idx]

    return (
        theoretical_r0 = reproduction_number(params),
        peak_infected = peak_infected,
        peak_fraction = peak_infected / initial_population,
        peak_time = global_df.time[peak_idx],
        final_recovered = global_df.recovered[end],
        deaths = initial_population - global_df.total[end],
    )
end

function beta_metrics(params::AbstractDict)
    model = initialize_sir(; pairs(params)...)
    peak_infected = 0.0
    for _ in 1:params[:n_steps]
        safe_step!(model)
        peak_infected = max(peak_infected, infected_fraction(model; denominator = sum(params[:Ns])))
    end

    initial_population = sum(params[:Ns])
    final_infected = infected_count(model) / initial_population
    final_recovered = recovered_count(model) / initial_population
    total_deaths = initial_population - total_count(model)

    return (
        peak = peak_infected,
        final_inf = final_infected,
        final_rec = final_recovered,
        deaths = total_deaths,
    )
end

function scan_beta_experiment(;
    beta_range = 0.1:0.1:1.0,
    seeds = [42, 43, 44],
    base_params = default_parameters(),
)
    results = NamedTuple[]

    for beta in beta_range
        for seed in seeds
            params = merge(
                copy(base_params),
                Dict(
                    :beta_und => fill(beta, length(base_params[:Ns])),
                    :beta_det => fill(beta / 10, length(base_params[:Ns])),
                    :seed => seed,
                ),
            )
            metrics = beta_metrics(params)
            push!(
                results,
                (
                    beta = beta,
                    seed = seed,
                    peak = metrics.peak,
```

```
                    final_inf = metrics.final_inf,
                    final_rec = metrics.final_rec,
                    deaths = metrics.deaths,
                ),
            )
        end
    end

    df = DataFrame(results)
    grouped = combine(
        groupby(df, :beta),
        :peak => mean => :mean_peak,
        :final_inf => mean => :mean_final_inf,
        :final_rec => mean => :mean_final_rec,
        :deaths => mean => :mean_deaths,
    )
    return df, grouped
end

function plot_beta_scan(grouped::DataFrame; population = 3000)
    plt = plot(
        grouped.beta,
        grouped.mean_peak;
        label = "Пик эпидемии",
        xlabel = "Коэффициент заразности beta",
        ylabel = "Доля населения",
        marker = :circle,
        linewidth = 2,
    )
    plot!(plt, grouped.beta, grouped.mean_final_inf; label = "Финальная доля I", marker = :square, linewidth = 2)
    plot!(plt, grouped.beta, grouped.mean_deaths ./ population; label = "Доля умерших", marker = :diamond, linewidth = 2)
    return plt
end

function threshold_study(;
    beta_range = 0.05:0.01:0.30,
    seeds = [42, 43, 44, 45, 46],
    base_params = default_parameters(),
    epidemic_threshold = 0.05,
)
    raw_df, grouped = scan_beta_experiment(; beta_range, seeds, base_params)
    observed_idx = findfirst(grouped.mean_peak .> epidemic_threshold)
    observed_beta = isnothing(observed_idx) ? missing : grouped.beta[observed_idx]
    theoretical_beta = 1 / base_params[:infection_period]

    summary = (
        epidemic_threshold = epidemic_threshold,
        observed_beta = observed_beta,
        theoretical_beta = theoretical_beta,
        threshold_gap = ismissing(observed_beta) ? missing : observed_beta - theoretical_beta,
    )

    return raw_df, grouped, summary
end

function plot_threshold_study(
    grouped::DataFrame;
    epidemic_threshold = 0.05,
```

```
    theoretical_beta = 1 / 14,
    observed_beta = missing,
)
    plt = plot(
        grouped.beta,
        grouped.mean_peak;
        xlabel = "beta",
        ylabel = "Средний пик доли I",
        linewidth = 2,
        marker = :circle,
        label = "Средний пик I",
    )
    hline!(plt, [epidemic_threshold]; linestyle = :dash, color = :firebrick, label = "Порог эпидемии 5%")
    vline!(plt, [theoretical_beta]; linestyle = :dashdot, color = :black, label = "Теоретический порог")
    if !ismissing(observed_beta)
        vline!(plt, [observed_beta]; linestyle = :dot, color = :seagreen, label = "Наблюдаемый порог")
    end
    return plt
end

function migration_metrics(params::AbstractDict)
    model = initialize_sir(; pairs(params)...)
    peak_value = 0.0
    peak_time = 0

    for step in 1:params[:n_steps]
        safe_step!(model)
        current_value = infected_fraction(model; denominator = sum(params[:Ns]))
        if current_value > peak_value
            peak_value = current_value
            peak_time = step
        end
    end

    return (peak_time = peak_time, peak_value = peak_value)
end

function scan_migration_experiment(;
    migration_intensities = 0.0:0.1:0.5,
    seeds = [42, 43, 44],
    base_params = default_parameters(),
)
    results = NamedTuple[]
    C = length(base_params[:Ns])

    for intensity in migration_intensities
        for seed in seeds
            params = merge(
                copy(base_params),
                Dict(
                    :migration_rates => create_migration_matrix(C, intensity),
                    :seed => seed,
                    :Is => [1, 0, 0],
                    :n_steps => 150,
                ),
            )
            metrics = migration_metrics(params)
            push!(
```

```
                results,
                (
                    migration_intensity = intensity,
                    seed = seed,
                    peak_time = metrics.peak_time,
                    peak_value = metrics.peak_value,
                ),
            )
        end
    end

    df = DataFrame(results)
    grouped = combine(
        groupby(df, :migration_intensity),
        :peak_time => mean => :mean_peak_time,
        :peak_value => mean => :mean_peak_value,
    )
    return df, grouped
end

function plot_migration_effect(grouped::DataFrame; population = 3000)
    p1 = plot(
        grouped.migration_intensity,
        grouped.mean_peak_time;
        marker = :circle,
        linewidth = 2,
        xlabel = "Интенсивность миграции",
        ylabel = "Время до пика, дни",
        label = "Время пика",
    )
    p2 = plot(
        grouped.migration_intensity,
        grouped.mean_peak_value .* population;
        marker = :square,
        linewidth = 2,
        xlabel = "Интенсивность миграции",
        ylabel = "Число инфицированных",
        label = "Пиковая заболеваемость",
    )
    return plot(p1, p2; layout = (2, 1), size = (900, 700))
end

function migration_summary(grouped::DataFrame)
    best_idx = argmin(grouped.mean_peak_time)
    best_row = grouped[best_idx, :]
    return (
        best_intensity = best_row.migration_intensity,
        best_peak_time = best_row.mean_peak_time,
        best_peak_value = best_row.mean_peak_value,
    )
end

function comprehensive_analysis_plot(df::DataFrame)
    grouped = combine(
        groupby(df, :beta),
        :peak => mean => :mean_peak,
        :final_inf => mean => :mean_final_inf,
        :final_rec => mean => :mean_final_rec,
```

```
        :deaths => mean => :mean_deaths,
    )

    p1 = plot(
        grouped.beta,
        grouped.mean_peak;
        label = "Пик",
        xlabel = "beta",
        ylabel = "Доля инфицированных",
        linewidth = 2,
    )
    plot!(p1, grouped.beta, grouped.mean_final_inf; label = "Финальная доля I", linewidth = 2)

    p2 = plot(
        grouped.beta,
        grouped.mean_deaths;
        label = "Умершие",
        xlabel = "beta",
        ylabel = "Число умерших",
        linewidth = 2,
        color = :firebrick,
    )

    p3 = plot(
        grouped.beta,
        grouped.mean_final_rec;
        label = "Доля выздоровевших",
        xlabel = "beta",
        ylabel = "Доля",
        linewidth = 2,
        color = :seagreen,
    )

    return plot(p1, p2, p3; layout = (3, 1), size = (900, 1000))
end

function scenario_parameters()
    return [
        Dict(
            :label => "baseline",
            :beta_und => [0.5, 0.5, 0.5],
            :beta_det => [0.05, 0.05, 0.05],
            :migration_intensity => 0.1,
            :detection_time => 7,
            :n_steps => 120,
            :seed => 42,
        ),
        Dict(
            :label => "heterogeneous_beta",
            :beta_und => [0.75, 0.45, 0.25],
            :beta_det => [0.08, 0.05, 0.03],
            :migration_intensity => 0.1,
            :detection_time => 7,
            :n_steps => 120,
            :seed => 52,
        ),
        Dict(
            :label => "fast_migration",
```

```
            :beta_und => [0.5, 0.5, 0.5],
            :beta_det => [0.05, 0.05, 0.05],
            :migration_intensity => 0.4,
            :detection_time => 7,
            :n_steps => 120,
            :seed => 62,
        ),
        Dict(
            :label => "slow_detection",
            :beta_und => [0.5, 0.5, 0.5],
            :beta_det => [0.05, 0.05, 0.05],
            :migration_intensity => 0.1,
            :detection_time => 10,
            :n_steps => 120,
            :seed => 72,
        ),
    ]
end

function run_scenario(spec::AbstractDict; base_params = default_parameters())
    params = merge(
        copy(base_params),
        Dict(
            :beta_und => spec[:beta_und],
            :beta_det => spec[:beta_det],
            :detection_time => spec[:detection_time],
            :seed => spec[:seed],
            :n_steps => spec[:n_steps],
            :migration_rates => create_migration_matrix(length(base_params[:Ns]), spec[:migration_intensity]),
        ),
    )

    model = initialize_sir(; pairs(params)...)
    global_df, city_df = collect_time_series(model, params[:n_steps])
    initial_population = sum(params[:Ns])
    peak_time = global_df.time[argmax(global_df.infected)]

    summary = (
        scenario = spec[:label],
        peak_infected = maximum(global_df.infected),
        peak_fraction = maximum(global_df.infected) / initial_population,
        peak_time = peak_time,
        final_recovered = global_df.recovered[end],
        deaths = initial_population - global_df.total[end],
    )

    return global_df, city_df, summary
end

function plot_scenario(global_df::DataFrame, city_df::DataFrame; title = "")
    p1 = plot(
        global_df.time,
        global_df.susceptible;
        label = "S",
        linewidth = 2,
        xlabel = "Дни",
        ylabel = "Количество",
        title = title,
```

```
    )
    plot!(p1, global_df.time, global_df.infected; label = "I", linewidth = 2)
    plot!(p1, global_df.time, global_df.recovered; label = "R", linewidth = 2)

    p2 = plot(
        city_df.time,
        city_df.city_1_infected;
        label = "Город 1",
        linewidth = 2,
        xlabel = "Дни",
        ylabel = "Инфицированные",
    )
    plot!(p2, city_df.time, city_df.city_2_infected; label = "Город 2", linewidth = 2)
    plot!(p2, city_df.time, city_df.city_3_infected; label = "Город 3", linewidth = 2)

    return plot(p1, p2; layout = (2, 1), size = (900, 700))
end

function heterogeneity_experiment(;
    base_params = default_parameters(),
    n_steps = 120,
    seed = 52,
)
    baseline_spec = Dict(
        :label => "homogeneous_beta",
        :beta_und => [0.5, 0.5, 0.5],
        :beta_det => [0.05, 0.05, 0.05],
        :migration_intensity => 0.1,
        :detection_time => 7,
        :n_steps => n_steps,
        :seed => seed,
    )
    hetero_spec = Dict(
        :label => "heterogeneous_beta",
        :beta_und => [0.80, 0.45, 0.20],
        :beta_det => [0.08, 0.05, 0.02],
        :migration_intensity => 0.1,
        :detection_time => 7,
        :n_steps => n_steps,
        :seed => seed,
    )

    base_global, base_city, base_summary = run_scenario(baseline_spec; base_params)
    hetero_global, hetero_city, hetero_summary = run_scenario(hetero_spec; base_params)

    city_summary = DataFrame(
        city = 1:3,
        beta_und = hetero_spec[:beta_und],
        peak_infected = [
            maximum(hetero_city[!, :city_1_infected]),
            maximum(hetero_city[!, :city_2_infected]),
            maximum(hetero_city[!, :city_3_infected]),
        ],
    )
    summary_df = DataFrame([base_summary, hetero_summary])

    return base_global, base_city, hetero_global, hetero_city, summary_df, city_summary
end
```

```
function plot_heterogeneity_global(base_global::DataFrame, hetero_global::DataFrame)
    plt = plot(
        base_global.time,
        base_global.infected;
        label = "Однородный beta",
        xlabel = "Дни",
        ylabel = "Число инфицированных",
        linewidth = 2,
    )
    plot!(plt, hetero_global.time, hetero_global.infected; label = "Неоднородный beta", linewidth = 2)
    return plt
end

function plot_city_dynamics(city_df::DataFrame; title = "Динамика по городам")
    plt = plot(
        city_df.time,
        city_df.city_1_infected;
        label = "Город 1",
        xlabel = "Дни",
        ylabel = "Инфицированные",
        linewidth = 2,
        title = title,
    )
    plot!(plt, city_df.time, city_df.city_2_infected; label = "Город 2", linewidth = 2)
    plot!(plt, city_df.time, city_df.city_3_infected; label = "Город 3", linewidth = 2)
    return plt
end

function quarantine_experiment(;
    thresholds = [0.05, 0.10, 0.15, 0.20],
    base_params = default_parameters(),
    beta = 0.20,
    migration_intensity = 0.10,
    n_steps = 120,
    seed = 91,
)
    results = NamedTuple[]
    dynamics = Dict{String, NamedTuple}()
    C = length(base_params[:Ns])

    scenarios = vcat([nothing], thresholds)
    for threshold in scenarios
        params = merge(
            copy(base_params),
            Dict(
                :beta_und => fill(beta, C),
                :beta_det => fill(beta / 10, C),
                :migration_rates => create_migration_matrix(C, migration_intensity),
                :Is => [1, 0, 0],
                :seed => seed,
                :n_steps => n_steps,
                :quarantine_threshold => threshold,
            ),
        )

        model = initialize_sir(; pairs(params)...)
        global_df, city_df = collect_time_series(model, params[:n_steps])
        initial_population = sum(params[:Ns])
```

```
        peak_idx = argmax(global_df.infected)
        label = isnothing(threshold) ? "no_quarantine" : "q_" * string(round(Int, threshold * 100))

        push!(
            results,
            (
                scenario = label,
                quarantine_threshold = something(threshold, -1.0),
                peak_fraction = global_df.infected[peak_idx] / initial_population,
                peak_time = global_df.time[peak_idx],
                deaths = initial_population - global_df.total[end],
                locked_cities = locked_city_count(model),
            ),
        )
        dynamics[label] = (global_df = global_df, city_df = city_df)
    end

    summary_df = DataFrame(results)
    return summary_df, dynamics
end

function plot_quarantine_summary(summary_df::DataFrame)
    labels = replace.(summary_df.scenario, "q_" => "q=")
    p1 = bar(
        labels,
        summary_df.peak_fraction;
        label = "Пиковая доля I",
        xlabel = "Сценарий",
        ylabel = "Доля",
    )
    hline!(p1, [0.30]; linestyle = :dash, color = :black, label = "Ограничение 30%")

    p2 = bar(
        labels,
        summary_df.deaths;
        label = "Смерти",
        xlabel = "Сценарий",
        ylabel = "Количество",
        color = :firebrick,
    )

    return plot(p1, p2; layout = (2, 1), size = (900, 700))
end

function plot_quarantine_dynamics(baseline_df::DataFrame, quarantine_df::DataFrame; quarantine_label = "")
    plt = plot(
        baseline_df.time,
        baseline_df.infected;
        label = "Без карантина",
        xlabel = "Дни",
        ylabel = "Инфицированные",
        linewidth = 2,
    )
    plot!(plt, quarantine_df.time, quarantine_df.infected; label = quarantine_label, linewidth = 2)
    return plt
end

function evaluate_policy(beta_und, detection_time, death_rate;
```

```
    base_params = default_parameters(),
    replicates = 5,
    n_steps = 100,
)
    peak_values = Float64[]
    death_values = Float64[]

    for rep in 1:replicates
        params = merge(
            copy(base_params),
            Dict(
                :beta_und => fill(beta_und, length(base_params[:Ns])),
                :beta_det => fill(beta_und / 10, length(base_params[:Ns])),
                :detection_time => detection_time,
                :death_rate => death_rate,
                :seed => 500 + rep,
                :n_steps => n_steps,
            ),
        )
        metrics = beta_metrics(params)
        push!(peak_values, metrics.peak)
        push!(death_values, metrics.deaths)
    end

    return mean(peak_values), mean(death_values)
end

function optimization_cost(x;
    base_params = default_parameters(),
    replicates = 5,
    n_steps = 100,
)
    beta_und = clamp(x[1], 0.1, 1.0)
    detection_time = clamp(round(Int, x[2]), 3, 14)
    death_rate = clamp(x[3], 0.01, 0.1)

    mean_peak, mean_deaths = evaluate_policy(
        beta_und,
        detection_time,
        death_rate;
        base_params,
        replicates,
        n_steps,
    )

    return (mean_peak, mean_deaths)
end

function pareto_frontier_dataframe(result, peak_limit)
    rows = NamedTuple[]

    for individual in pareto_frontier(result)
        candidate = params(individual)
        objectives = archived_fitness(individual)
        objective_tuple = objectives.orig
        detection_time = clamp(round(Int, candidate[2]), 3, 14)

        push!(
```

```
            rows,
            (
                beta_und = candidate[1],
                detection_time = detection_time,
                death_rate = candidate[3],
                mean_peak = objective_tuple[1],
                mean_deaths = objective_tuple[2],
                feasible = objective_tuple[1] <= peak_limit,
            ),
        )
    end

    frontier_df = DataFrame(rows)
    nrow(frontier_df) == 0 && return frontier_df

    sort!(frontier_df, [:mean_peak, :mean_deaths, :beta_und, :detection_time, :death_rate])
    unique!(frontier_df)

    return frontier_df
end

function optimize_under_peak_constraint(;
    peak_limit = 0.30,
    max_evals = 30,
    replicates = 2,
    n_steps = 60,
    seed = 2026,
    base_params = default_parameters(),
)
    Random.seed!(seed)

    result = bboptimize(
        x -> optimization_cost(
            x;
            base_params,
            replicates,
            n_steps,
        ),
        Method = :borg_moea,
        FitnessScheme = ParetoFitnessScheme{2}(is_minimizing = true),
        SearchRange = [(0.1, 1.0), (3.0, 14.0), (0.01, 0.1)],
        NumDimensions = 3,
        MaxFuncEvals = max_evals,
        TraceMode = :compact,
    )

    all_df = pareto_frontier_dataframe(result, peak_limit)
    feasible_df = filter(:feasible => identity, all_df)
    sort!(feasible_df, [:mean_deaths, :mean_peak, :beta_und, :detection_time, :death_rate])

    best = nrow(feasible_df) == 0 ? nothing : feasible_df[1, :]
    return all_df, feasible_df, best
end

function plot_constrained_optimization(all_df::DataFrame, feasible_df::DataFrame; peak_limit = 0.30)
    plt = scatter(
        all_df.mean_peak,
        all_df.mean_deaths;
```

```
        xlabel = "Средний пик доли I",
        ylabel = "Среднее число умерших",
        label = "Точки Парето-фронта",
        alpha = 0.65,
        markerstrokewidth = 0,
        color = :steelblue,
    )
    if nrow(feasible_df) > 0
        scatter!(plt, feasible_df.mean_peak, feasible_df.mean_deaths; label = "Допустимые", color = :darkgreen)
    end
    vline!(plt, [peak_limit]; linestyle = :dash, color = :firebrick, label = "Предел 30%")
    return plt
end

function scenario_comparison_plot(summary_df::DataFrame)
    p1 = bar(
        summary_df.scenario,
        summary_df.peak_fraction;
        label = "Пиковая доля I",
        xlabel = "Сценарий",
        ylabel = "Доля",
        legend = :topright,
    )
    p2 = bar(
        summary_df.scenario,
        summary_df.deaths;
        label = "Число умерших",
        xlabel = "Сценарий",
        ylabel = "Количество",
        color = :firebrick,
        legend = :topright,
    )
    return plot(p1, p2; layout = (2, 1), size = (900, 700))
end
```

6.3 Генератор literate-форматов `generate.jl`

```
using DrWatson
@quickactivate "lab_04_models"
using Literate

scripts_dir = @__DIR__
notebooks_dir = joinpath(@__DIR__, "..", "notebooks")
docs_dir = joinpath(@__DIR__, "..", "docs")

mkpath(notebooks_dir)
mkpath(docs_dir)

function quarto_header(title, description)
    return """
---
title: "$(title)"
description: "$(description)"
```

```
lang: ru-RU
jupyter: julia-1.12
format:
  html:
    toc: true
    number-sections: true
    code-fold: false
execute:
  cache: true
---

"""
end

sources = [
    (
        "sir_literate.jl",
        "sir",
        "SIR model on a graph of cities",
        "Base experiments for the graph-based SIR model",
    ),
    (
        "sir_param_literate.jl",
        "sir_param",
        "SIR model: parameter scenarios",
        "Scenario analysis for the graph-based SIR model",
    ),
    (
        "sir_threshold_literate.jl",
        "sir_threshold",
        "SIR model: threshold study",
        "Threshold study for the graph-based SIR model",
    ),
    (
        "sir_heterogeneity_literate.jl",
        "sir_heterogeneity",
        "SIR model: heterogeneity effect",
        "Heterogeneity study for the graph-based SIR model",
    ),
    (
        "sir_quarantine_literate.jl",
        "sir_quarantine",
        "SIR model: quarantine policy",
        "Quarantine policy analysis for the graph-based SIR model",
    ),
    (
        "sir_optimize_literate.jl",
        "sir_optimize",
        "SIR model: constrained optimization",
        "Constrained optimization for the graph-based SIR model",
    ),
]

for (src_file, base, title, description) in sources
    src_path = joinpath(scripts_dir, src_file)

    Literate.script(src_path, scripts_dir; name = base * "_clean", credit = false)
    Literate.notebook(src_path, notebooks_dir; name = base, execute = true, credit = false)
```

```
    Literate.markdown(src_path, docs_dir; name = base, flavor = Literate.QuartoFlavor(), credit = false)

    qmd_path = joinpath(docs_dir, base * ".qmd")
    content = read(qmd_path, String)
    write(qmd_path, quarto_header(title, description) * content)
end

println("Literate generation completed.")
```

6.4 Скрипт базового эксперимента sir_run_basic.jl

```
using DrWatson
@quickactivate "lab_04_models"

ENV["GKSwstype"] = "100"

include(srcdir("sir_analysis.jl"))

params = default_parameters()
agent_df, model_df = run_basic_experiment(params)
basic_plot = plot_basic_dynamics(agent_df, model_df)
summary_df = DataFrame([basic_summary(params)])

savefig(basic_plot, plotsdir("sir_basic_dynamics.png"))
@save datadir("sir_basic_agent.jld2") agent_df
@save datadir("sir_basic_model.jld2") model_df
CSV.write(datadir("basic_summary.csv"), summary_df)

println("Basic experiment completed.")
```

6.5 Скрипт сканирования beta sir_scan_beta.jl

```
using DrWatson
@quickactivate "lab_04_models"

ENV["GKSwstype"] = "100"

include(srcdir("sir_analysis.jl"))

beta_df, beta_grouped = scan_beta_experiment()
beta_plot = plot_beta_scan(beta_grouped)

CSV.write(datadir("beta_scan_all.csv"), beta_df)
savefig(beta_plot, plotsdir("beta_scan.png"))
```

```
println("Beta scan completed.")
```

6.6 Скрипт исследования миграции sir_migration_effect.jl

```
using DrWatson
@quickactivate "lab_04_models"

ENV["GKSwstype"] = "100"

include(srcdir("sir_analysis.jl"))

migration_df, migration_grouped = scan_migration_experiment()
migration_plot = plot_migration_effect(migration_grouped)

CSV.write(datadir("migration_scan_all.csv"), migration_df)
savefig(migration_plot, plotsdir("migration_effect.png"))

println("Migration scan completed.")
```

6.7 Скрипт итоговой визуализации sir_visualize_dynamics.jl

```
using DrWatson
@quickactivate "lab_04_models"

ENV["GKSwstype"] = "100"

include(srcdir("sir_analysis.jl"))

beta_df = CSV.read(datadir("beta_scan_all.csv"), DataFrame)
comprehensive_plot = comprehensive_analysis_plot(beta_df)

savefig(comprehensive_plot, plotsdir("comprehensive_analysis.png"))

println("Comprehensive plot completed.")
```

6.8 Скрипт порогового исследования sir_threshold_study.jl

```
using DrWatson
@quickactivate "lab_04_models"

ENV["GKSwstype"] = "100"

include(srcdir("sir_analysis.jl"))

threshold_df, threshold_grouped, threshold_summary = threshold_study()
threshold_plot = plot_threshold_study(
    threshold_grouped;
    epidemic_threshold = threshold_summary.epidemic_threshold,
    theoretical_beta = threshold_summary.theoretical_beta,
    observed_beta = threshold_summary.observed_beta,
)

CSV.write(datadir("threshold_scan_all.csv"), threshold_df)
CSV.write(datadir("threshold_scan_grouped.csv"), threshold_grouped)
CSV.write(datadir("threshold_summary.csv"), DataFrame([threshold_summary]))
savefig(threshold_plot, plotsdir("threshold_study.png"))

println("Threshold study completed.")
```

6.9 Скрипт исследования гетерогенности sir_heterogeneity_effect.jl

```
using DrWatson
@quickactivate "lab_04_models"

ENV["GKSwstype"] = "100"

include(srcdir("sir_analysis.jl"))

base_global, base_city, hetero_global, hetero_city, summary_df, city_summary = heterogeneity_experiment()

global_plot = plot_heterogeneity_global(base_global, hetero_global)
city_plot = plot_city_dynamics(hetero_city; title = "Неоднородный beta по городам")
combined_plot = plot(global_plot, city_plot; layout = (2, 1), size = (900, 800))

CSV.write(datadir("heterogeneity_summary.csv"), summary_df)
CSV.write(datadir("heterogeneity_city_summary.csv"), city_summary)
savefig(global_plot, plotsdir("heterogeneity_global_comparison.png"))
savefig(city_plot, plotsdir("heterogeneity_city_dynamics.png"))
savefig(combined_plot, plotsdir("heterogeneity_effect.png"))
```

```
println("Heterogeneity experiment completed.")
```

6.10 Скрипт карантинного сценария sir_quarantine.jl

```
using DrWatson
@quickactivate "lab_04_models"

ENV["GKSwstype"] = "100"

include(srcdir("sir_analysis.jl"))

summary_df, dynamics = quarantine_experiment()
summary_plot = plot_quarantine_summary(summary_df)

candidate_df = filter(row -> row.scenario != "no_quarantine" && row.locked_cities > 0, summary_df)
best_label = if nrow(candidate_df) == 0
    "no_quarantine"
else
    candidate_df[argmin(candidate_df.deaths), :scenario]
end

baseline_df = dynamics["no_quarantine"].global_df
best_df = dynamics[best_label].global_df
dynamics_plot = plot_quarantine_dynamics(baseline_df, best_df; quarantine_label = best_label)

CSV.write(datadir("quarantine_summary.csv"), summary_df)
savefig(summary_plot, plotsdir("quarantine_effect.png"))
savefig(dynamics_plot, plotsdir("quarantine_best_dynamics.png"))

println("Quarantine experiment completed.")
```

6.11 Скрипт оптимизации sir_optimize_parameters.jl

```
using DrWatson
@quickactivate "lab_04_models"

ENV["GKSwstype"] = "100"

include(srcdir("sir_analysis.jl"))

peak_limit = 0.30
```

```
all_df, feasible_df, best = optimize_under_peak_constraint(; peak_limit)
opt_plot = plot_constrained_optimization(all_df, feasible_df; peak_limit)

@save datadir("optimization_result.jld2") best feasible_df all_df
CSV.write(datadir("optimization_samples.csv"), all_df)
CSV.write(datadir("optimization_pareto.csv"), feasible_df)
savefig(opt_plot, plotsdir("optimization_pareto.png"))

println("Optimization completed.")
if best === nothing
    println("No feasible solution found under peak limit $(peak_limit).")
else
    println("Best feasible candidate:")
    println("  beta_und            = $(round(best.beta_und, digits = 4))")
    println("  detection_time      = $(best.detection_time)")
    println("  death_rate          = $(round(best.death_rate, digits = 4))")
    println("  mean_peak           = $(round(best.mean_peak, digits = 4))")
    println("  mean_deaths         = $(round(best.mean_deaths, digits = 2))")
end
```

6.12 Literate-источник `sir_literate.jl`

```
# # SIR model on a graph of cities
#
# This literate script reproduces the main experiments for laboratory work 4:
# a base SIR run, a scan over transmission rate, a migration scan,
# and a summary visualization.
#
# ## Libraries and project activation

using DrWatson
@quickactivate "lab_04_models"

ENV["GKSwstype"] = "100"

include(srcdir("sir_analysis.jl"))

# ## Base experiment
#
# We start with three cities of equal population and seed one infected agent
# in the third city.

params = default_parameters()
agent_df, model_df = run_basic_experiment(params)
basic_plot = plot_basic_dynamics(agent_df, model_df)
basic_plot

#jl savefig(basic_plot, plotsdir("sir_basic_dynamics.png"))
#jl @save datadir("sir_basic_agent.jld2") agent_df
#jl @save datadir("sir_basic_model.jld2") model_df

# ## Scan over transmission intensity
#
```

```
# The next experiment varies `beta` from `0.1` to `1.0`
# and averages the result over three random seeds.

beta_df, beta_grouped = scan_beta_experiment()
beta_plot = plot_beta_scan(beta_grouped)
beta_plot

#jl CSV.write(datadir("beta_scan_all.csv"), beta_df)
#jl savefig(beta_plot, plotsdir("beta_scan.png"))

# ## Migration effect
#
# We vary the migration intensity and observe how it changes the time to peak
# and the peak number of infected agents.

migration_df, migration_grouped = scan_migration_experiment()
migration_plot = plot_migration_effect(migration_grouped)
migration_plot

#jl CSV.write(datadir("migration_scan_all.csv"), migration_df)
#jl savefig(migration_plot, plotsdir("migration_effect.png"))

# ## Summary plot
#
# The final figure combines three panels built from the beta scan:
# peak infection, deaths, and the final recovered fraction.

comprehensive_plot = comprehensive_analysis_plot(beta_df)
comprehensive_plot

#jl savefig(comprehensive_plot, plotsdir("comprehensive_analysis.png"))

# ## Conclusions
#
# 1. The base run shows a standard epidemic wave with a clear infection peak.
# 2. The threshold behavior becomes visible in the beta scan.
# 3. Higher migration moves the infection peak earlier and makes it stronger.
# 4. The combined figure is suitable for the lab report and presentation.
```

6.13 Literate-источник `sir_param_literate.jl`

```
# # Parameter scenarios for the graph SIR model
#
# This script extends the base experiment with several parameter scenarios:
# heterogeneous transmission, faster migration, and slower detection.
#
# ## Libraries and project activation

using DrWatson
@quickactivate "lab_04_models"

ENV["GKSwstype"] = "100"
```

```julia
include(srcdir("sir_analysis.jl"))

# ## Scenario list
#
# Each scenario modifies a small set of parameters while keeping the model
# structure unchanged.

scenarios = scenario_parameters()
summaries = NamedTuple[]

for spec in scenarios
    global_df, city_df, summary = run_scenario(spec)
    scenario_plot = plot_scenario(global_df, city_df; title = spec[:label])
    scenario_plot
    plot_name = "scenario_" * spec[:label] * ".png"

#jl     savefig(scenario_plot, plotsdir(plot_name))

    push!(summaries, summary)
end

summary_df = DataFrame(summaries)
summary_df

#jl CSV.write(datadir("scenario_summary.csv"), summary_df)

# ## Scenario comparison
#
# We compare peak infection fractions and death counts across all scenarios.

comparison_plot = scenario_comparison_plot(summary_df)
comparison_plot

#jl savefig(comparison_plot, plotsdir("scenario_comparison.png"))

# ## Conclusions
#
# 1. Heterogeneous transmission shifts the infection burden toward the first city.
# 2. Faster migration reduces the time to epidemic peak.
# 3. Slower detection increases both the peak load and the number of deaths.
```

6.14 Literate-источник `sir_threshold_literate.jl`

```julia
# # SIR model: threshold study
#
# Этот literate-скрипт исследует эпидемический порог в агентной SIR-модели.
# Мы хотим найти минимальное значение `beta`, при котором средний пик
# инфицированных превышает `5%` популяции.

using DrWatson
@quickactivate "lab_04_models"

ENV["GKSwstype"] = "100"
```

```julia
include(srcdir("sir_analysis.jl"))

# ## Плотное сканирование коэффициента заражения
#
# В отличие от базового `sir_scan_beta.jl` здесь используется более мелкий шаг
# по `beta`, чтобы аккуратно сравнить наблюдаемый порог с теоретическим
# условием `R0 = 1`.

threshold_df, threshold_grouped, threshold_summary = threshold_study()
threshold_grouped

# ## График порога
#
# На графике показываются:
# - средний пик инфицированных;
# - эмпирический критерий эпидемии `5%`;
# - теоретический порог;
# - наблюдаемый порог по результатам моделирования.

threshold_plot = plot_threshold_study(
    threshold_grouped;
    epidemic_threshold = threshold_summary.epidemic_threshold,
    theoretical_beta = threshold_summary.theoretical_beta,
    observed_beta = threshold_summary.observed_beta,
)
threshold_plot

#jl CSV.write(datadir("threshold_scan_all.csv"), threshold_df)
#jl CSV.write(datadir("threshold_scan_grouped.csv"), threshold_grouped)
#jl CSV.write(datadir("threshold_summary.csv"), DataFrame([threshold_summary]))
#jl savefig(threshold_plot, plotsdir("threshold_study.png"))

# ## Выводы
#
# 1. Теоретический и наблюдаемый пороги не совпадают буквально.
# 2. В агентной стохастической модели вспышка должна не только начаться,
#    но и превысить критерий `5%`.
# 3. Поэтому эмпирический порог получается выше классической оценки `R0 = 1`.
```

6.15 Literate-источник sir_heterogeneity_literate.jl

```julia
# # SIR model: heterogeneity effect
#
# Этот literate-скрипт исследует влияние неоднородных коэффициентов заражения
# по городам. Сравниваются однородный и неоднородный режимы передачи инфекции.

using DrWatson
@quickactivate "lab_04_models"
```

```julia
ENV["GKSwstype"] = "100"

include(srcdir("sir_analysis.jl"))

# ## Сравнение глобальной динамики
#
# Функция `heterogeneity_experiment()` запускает два сценария:
# - однородный `beta`;
# - неоднородный `beta` по трём городам.

base_global, base_city, hetero_global, hetero_city, summary_df, city_summary =
    heterogeneity_experiment()

summary_df

global_plot = plot_heterogeneity_global(base_global, hetero_global)
global_plot

#jl savefig(global_plot, plotsdir("heterogeneity_global_comparison.png"))

# ## Динамика по городам
#
# Здесь нас интересует, как распределяется инфекционная нагрузка между городами.

city_plot = plot_city_dynamics(hetero_city; title = "Неоднородный beta по городам")
city_plot

#jl savefig(city_plot, plotsdir("heterogeneity_city_dynamics.png"))

# ## Составной график

combined_plot = plot(global_plot, city_plot; layout = (2, 1), size = (900, 800))
combined_plot

#jl CSV.write(datadir("heterogeneity_summary.csv"), summary_df)
#jl CSV.write(datadir("heterogeneity_city_summary.csv"), city_summary)
#jl savefig(combined_plot, plotsdir("heterogeneity_effect.png"))

# ## Выводы
#
# 1. Неоднородные значения `beta` меняют распределение нагрузки между городами.
# 2. Из-за миграции различия не изолируются в одном городе.
# 3. Суммарная смертность может даже вырасти, несмотря на неодинаковые параметры.
```

6.16 Literate-источник `sir_quarantine_literate.jl`

```julia
# # SIR model: quarantine policy
#
# Этот literate-скрипт исследует карантинную меру: город закрывается,
# если доля инфицированных в нём превышает заданный порог.

using DrWatson
@quickactivate "lab_04_models"
```

```julia
ENV["GKSwstype"] = "100"

include(srcdir("sir_analysis.jl"))

# ## Серия карантинных порогов
#
# Сценарий сравнивает режим без карантина с несколькими значениями
# `quarantine_threshold`.

summary_df, dynamics = quarantine_experiment()
summary_df

summary_plot = plot_quarantine_summary(summary_df)
summary_plot

#jl CSV.write(datadir("quarantine_summary.csv"), summary_df)
#jl savefig(summary_plot, plotsdir("quarantine_effect.png"))

# ## Сравнение лучшего карантинного режима с базовым
#
# Из всех карантинных сценариев выбирается режим с минимальным числом смертей.

candidate_df = filter(row -> row.scenario != "no_quarantine" && row.locked_cities > 0, summary_df)
best_label = if nrow(candidate_df) == 0
    "no_quarantine"
else
    candidate_df[argmin(candidate_df.deaths), :scenario]
end

baseline_df = dynamics["no_quarantine"].global_df
best_df = dynamics[best_label].global_df
dynamics_plot = plot_quarantine_dynamics(baseline_df, best_df; quarantine_label = best_label)
dynamics_plot

#jl savefig(dynamics_plot, plotsdir("quarantine_best_dynamics.png"))

# ## Выводы
#
# 1. Закрытие города уменьшает исходящую миграцию и замедляет распространение.
# 2. Разные пороги дают разный компромисс между скоростью пика и смертностью.
# 3. В проверенной серии лучший эффект дал самый ранний из рассмотренных порогов.
```

6.17 Literate-источник `sir_optimize_literate.jl`

```julia
# # SIR model: constrained optimization
#
# Этот literate-скрипт решает задачу оптимизации: найти такие параметры,
# которые минимизируют число умерших при ограничении на средний пик
# заболеваемости `mean_peak <= 30%`.

using DrWatson
@quickactivate "lab_04_models"
```

```julia
ENV["GKSwstype"] = "100"

include(srcdir("sir_analysis.jl"))

# ## Поиск допустимых решений
#
# Используется многокритериальная оптимизация `BlackBoxOptim` (`:borg_moea`).
# Оптимизируются параметры:
# - `beta_und`;
# - `detection_time`;
# - `death_rate`.

peak_limit = 0.30
all_df, feasible_df, best = optimize_under_peak_constraint(; peak_limit)

first(feasible_df, min(10, nrow(feasible_df)))

# ## График допустимых решений

opt_plot = plot_constrained_optimization(all_df, feasible_df; peak_limit)
opt_plot

#jl @save datadir("optimization_result.jld2") best feasible_df all_df
#jl CSV.write(datadir("optimization_samples.csv"), all_df)
#jl CSV.write(datadir("optimization_pareto.csv"), feasible_df)
#jl savefig(opt_plot, plotsdir("optimization_pareto.png"))

# ## Выводы
#
# 1. Алгоритм строит Pareto-фронт по двум критериям: средний пик и среднее число умерших.
# 2. Ограничение `mean_peak <= 30%` позволяет выбрать допустимые решения из фронта.
# 3. Лучшие решения смещаются к меньшей заразности, более раннему выявлению и меньшей смертности.
```

6.18 Тесты runtests.jl

```julia
using DrWatson
using Test

@quickactivate "lab_04_models"

include(srcdir("sir_model.jl"))

println("Starting tests")

@testset "lab_04_models tests" begin
    migration_matrix = create_migration_matrix(3, 0.2)
    @test all(isapprox(sum(migration_matrix[i, :]), 1.0; atol = 1e-8) for i in 1:3)
    @test all(isapprox(migration_matrix[i, i], 0.8; atol = 1e-8) for i in 1:3)

    model = initialize_sir(
        Ns = [10, 10, 10],
        beta_und = [0.0, 0.0, 0.0],
```

```
        beta_det = [0.0, 0.0, 0.0],
        infection_period = 3,
        detection_time = 1,
        death_rate = 0.0,
        reinfection_probability = 0.0,
        Is = [0, 0, 2],
        migration_rates = create_migration_matrix(3, 0.0),
        seed = 7,
        n_steps = 5,
    )

    @test total_count(model) == 30
    @test infected_count(model) == 2
    @test susceptible_count(model) == 28
    @test recovered_count(model) == 0

    safe_step!(model, 3)
    @test infected_count(model) == 0
    @test recovered_count(model) == 2
    @test total_count(model) == 30

    quarantine_model = initialize_sir(
        Ns = [10, 10, 10],
        beta_und = [0.0, 0.0, 0.0],
        beta_det = [0.0, 0.0, 0.0],
        infection_period = 5,
        detection_time = 1,
        death_rate = 0.0,
        reinfection_probability = 0.0,
        Is = [1, 0, 0],
        migration_rates = create_migration_matrix(3, 0.2),
        quarantine_threshold = 0.01,
        seed = 11,
        n_steps = 3,
    )

    safe_step!(quarantine_model, 1)
    @test quarantine_model.city_locked_down[1]
    @test quarantine_model.migration_rates[1, 1] == 1.0
    @test sum(quarantine_model.migration_rates[1, :]) == 1.0
end
```

7 Выводы

В ходе лабораторной работы была реализована агентная эпидемиологическая SIR-модель на графе трёх городов. Проект оформлен в структуре DrWatson, основная логика вынесена в `src/sir_model.jl` и `src/sir_analysis.jl`, а для воспроизводимости экспериментов использованы отдельные сценарные скрипты.

Были выполнены все основные и дополнительные задания: базовый эксперимент, сканирование коэффициента заразности, исследование миграции, сценарный анализ, оценка `R_0`, поиск порога эпидемии, анализ гетерогенности, карантинные меры и оптимизация при ограничении на пик. Все ключевые сценарии оформлены также в literate-стиле и преобразованы в `.jl`, `.ipynb` и `.qmd`.

Полученные результаты содержательно согласуются с теорией. Базовый режим с `R_0 = 7` приводит к мощной вспышке, рост `beta` резко усиливает эпидемию, межгородская миграция ускоряет распространение, неоднородность перераспределяет локальные пики, а карантин и раннее выявление позволяют смягчить последствия. Наиболее эффективным в исследованном диапазоне карантинных порогов оказался режим `q_5`, а оптимизация при ограничении на пик показала ожидаемый переход к слабому распространению инфекции.

Список литературы

1. *Bonabeau E.* Agent-based modeling: Methods and techniques for simulating human systems // Proceedings of the National Academy of Sciences. — 2002. — Vol. 99, suppl 3. — P. 7280–7287. — DOI: 10.1073/pnas.082080899.
2. *Datseris G.* DrWatson.jl: the perfect sidekick to your scientific inquiries. — 2020. — URL: https://juliadynamics.github.io/DrWatson.jl/stable/ ; Accessed: 2026-04-04.
3. *Datseris G.*, *Vahdati A. R.*, *DuBois T. C.* Agents.jl: a performant and feature-full agent-based modeling software of minimal code complexity // SIMULATION. — 2022. — Vol. 98, no. 4. — P. 313–327. — DOI: 10.1177/00375497211068820.
4. *Ekre F.*, *contributors.* Literate.jl. — 2026. — URL: https://fredrikekre.github.io/Literate.jl/ ; Accessed: 2026-04-04.
5. *Hethcote H. W.* The Mathematics of Infectious Diseases // SIAM Review. — 2000. — Vol. 42, no. 4. — P. 599–653. — DOI: 10.1137/S0036144500371907.
6. *Kermack W. O.*, *McKendrick A. G.* A contribution to the mathematical theory of epidemics // Proceedings of the Royal Society A. — 1927. — Vol. 115, no. 772. — P. 700–721. — DOI: 10.1098/rspa.1927.0118.
7. *Knuth D. E.* Literate Programming // The Computer Journal. — 1984. — Vol. 27, no. 2. — P. 97–111. — DOI: 10.1093/comjnl/27.2.97.
8. *Macal C. M.*, *North M. J.* Tutorial on agent-based modelling and simulation // Journal of Simulation. — 2010. — Vol. 4, no. 3. — P. 151–162. — DOI: 10.1057/jos.2010.3.